

アルゴリズム戦略

今日学ぶこと

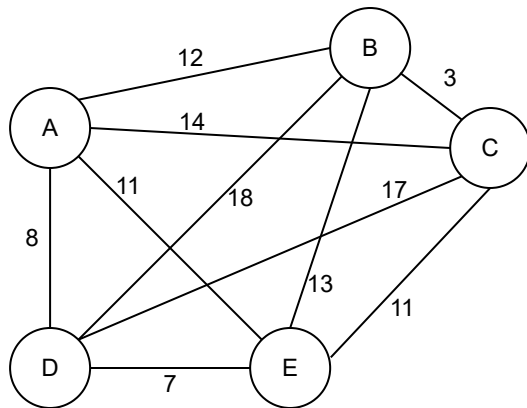
良くあるアルゴリズムの作り方と例

- パラダイム (考え方)
 - 総当たり法
 - 分割統治法
 - 貪欲法
 - 問題の変換
 - 時間と空間のトレードオフ
 - 近似アルゴリズム
 - 逐次改善法
 - 確率的アルゴリズム
- 指数的爆発対策
- ループと再帰

総当たり法

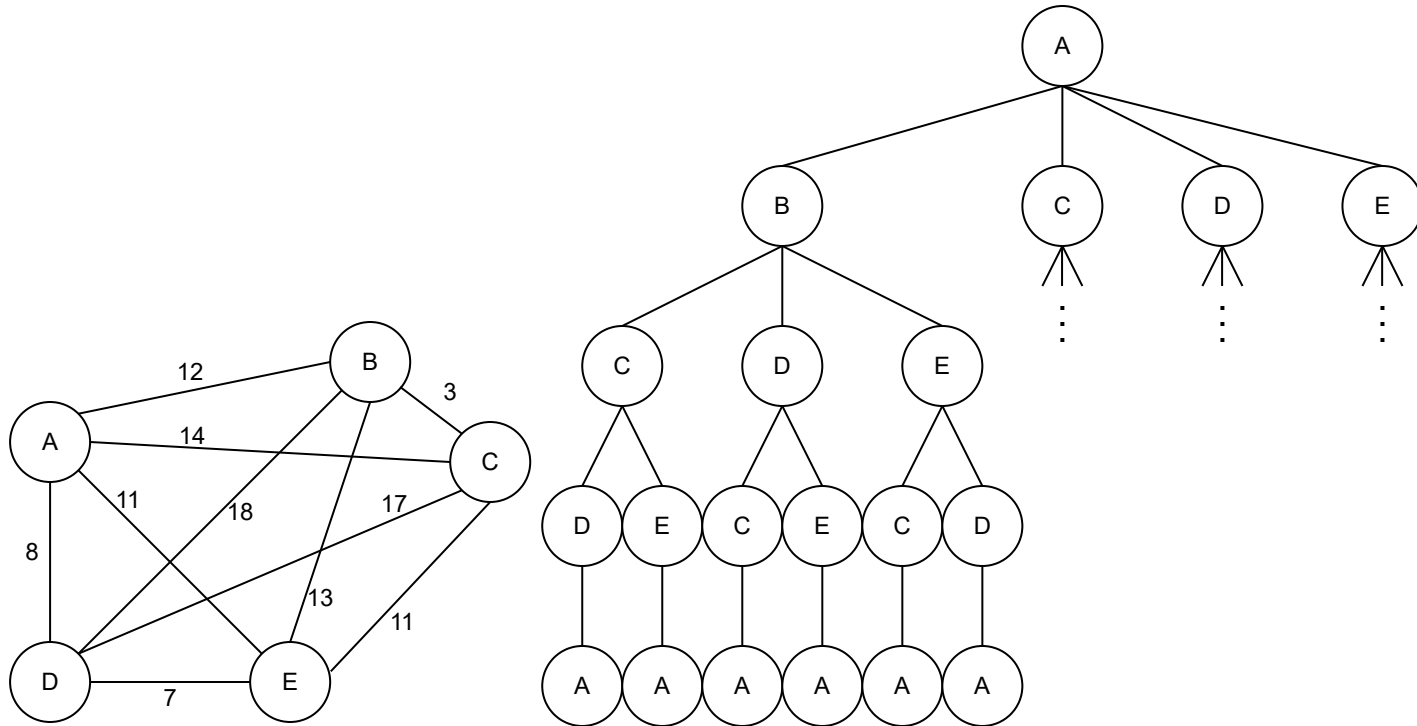
巡回セールスマン問題

n個の都市を全部1回ずつ訪れて戻ってくる最短距離を求めたい。



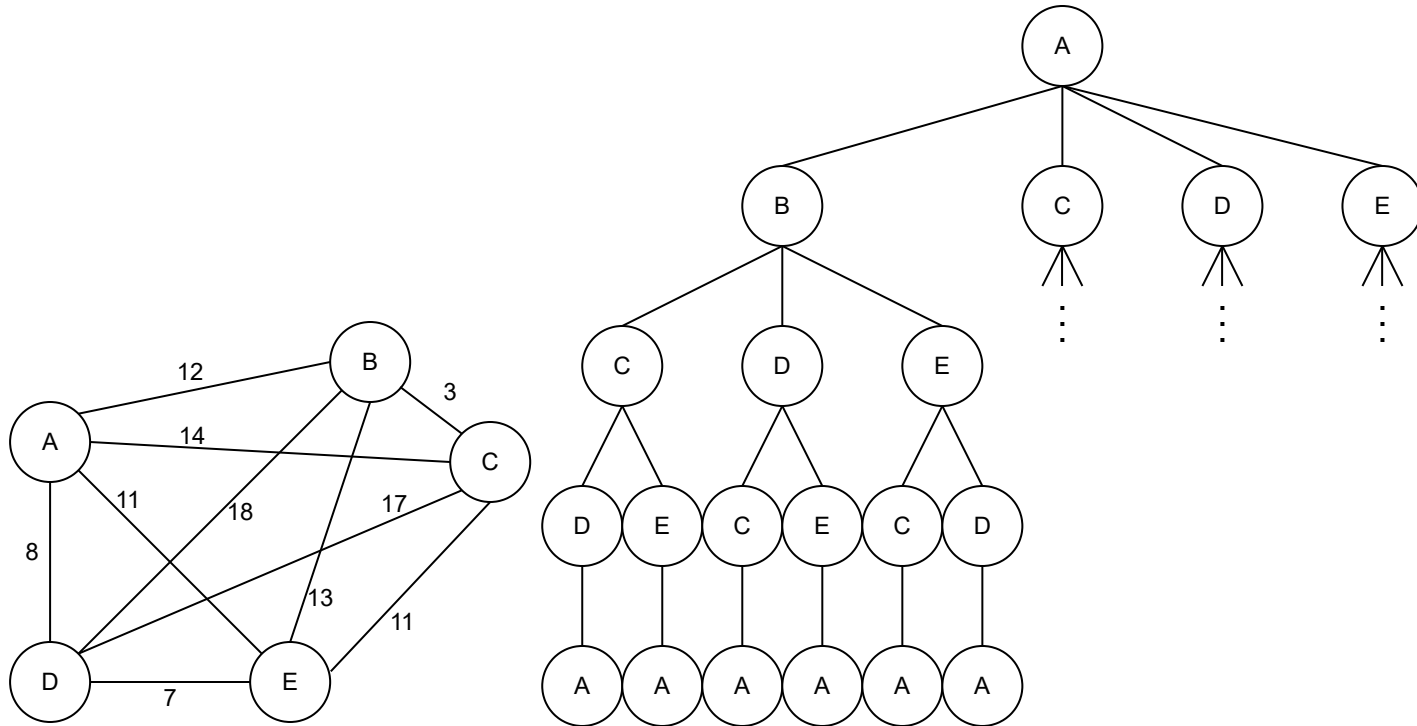
巡回セールスマン問題

n個の都市を全部1回ずつ訪れて戻ってくる最短距離を求めたい。



巡回セールスマン問題

n個の都市を全部1回ずつ訪れて戻ってくる最短距離を求めたい。



計算量は $O(n!)$ ($n! = n \times (n-1) \times \dots \times 1$)

ナップサック問題

重さが w 以下で価値の総計 v が最大になるように、 n 個の品物からいくつか選びたい。

(合計 w を 5 以下にしたい)

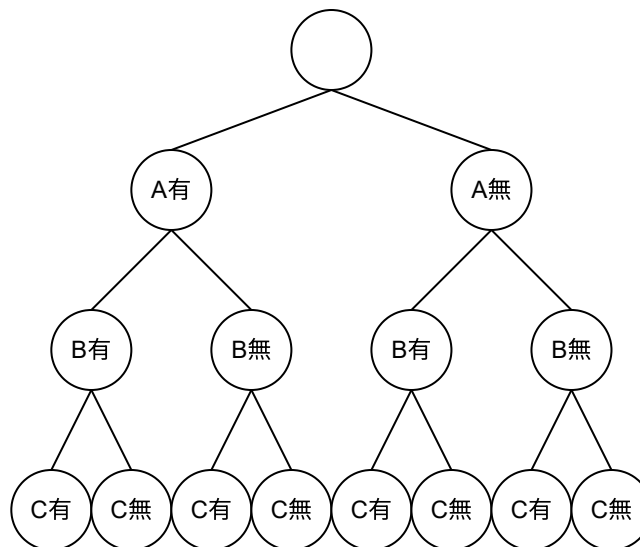
	w	v
A. ゲーム機	5	10
B. 積み木	3	4
C. 指輪	1	8

ナップサック問題

重さが w 以下で価値の総計 v が最大になるように、 n 個の品物からいくつか選びたい。

(合計 w を5以下にしたい)

	w	v
A. ゲーム機	5	10
B. 積み木	3	4
C. 指輪	1	8

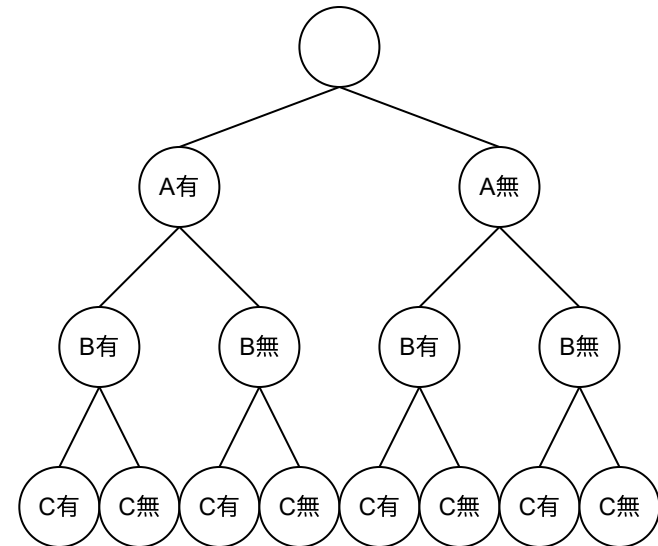


ナップサック問題

重さが w 以下で価値の総計 v が最大になるように、 n 個の品物からいくつか選びたい。

(合計 w を5以下にしたい)

	w	v
A. ゲーム機	5	10
B. 積み木	3	4
C. 指輪	1	8



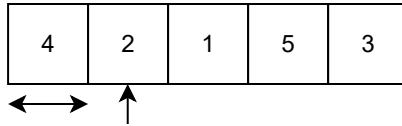
計算量は $O(2^n)$

分割統治法

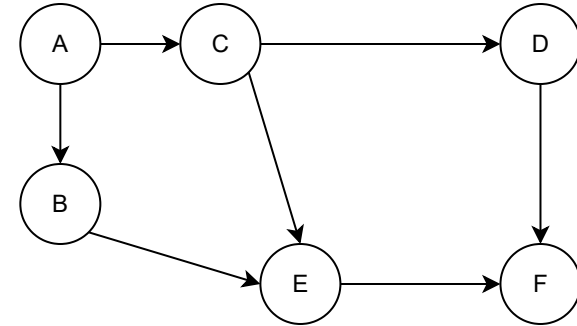
少しずつ問題を小さくしていく (DECREASE AND CONQUER)

(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



トポロジカルソート (TOPOLOGICAL SORT)

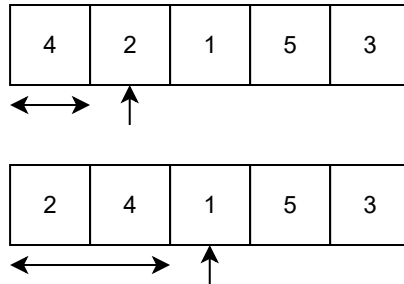


残り4つの整数から1つ取り、ソート済み配列にマージ。

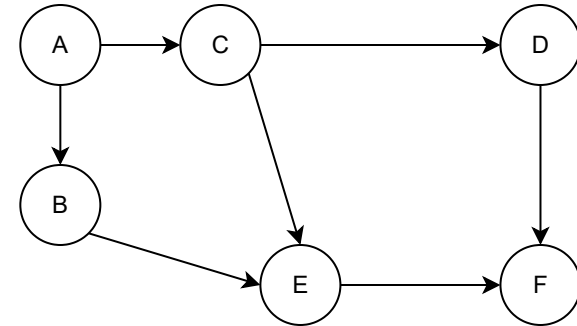
少しずつ問題を小さくしていく (DECREASE AND CONQUER)

(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



トポロジカルソート (TOPOLOGICAL SORT)

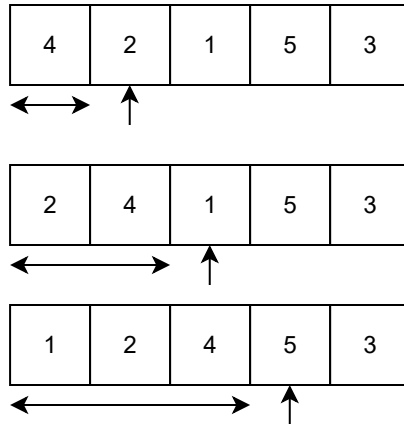


残り4つの整数から1つ取り、ソート済み配列にマージ。
残り3つの整数から1つ取り、ソート済み配列にマージ。

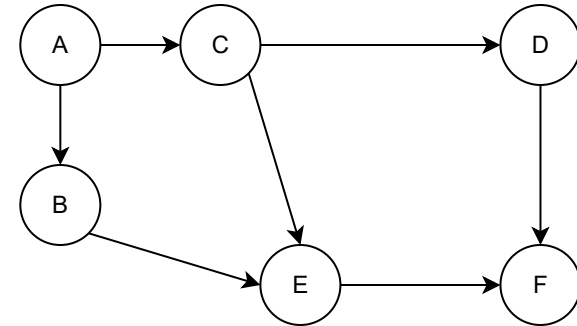
少しずつ問題を小さくしていく (DECREASE AND CONQUER)

(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



トポロジカルソート (TOPOLOGICAL SORT)

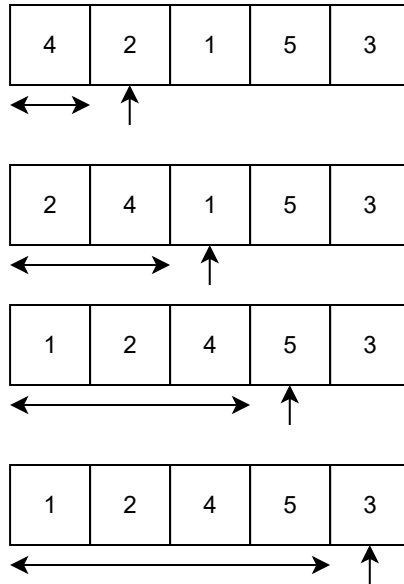


残り4つの整数から1つ取り、ソート済み配列にマージ。
残り3つの整数から1つ取り、ソート済み配列にマージ。
残り2つの整数から1つ取り、ソート済み配列にマージ。

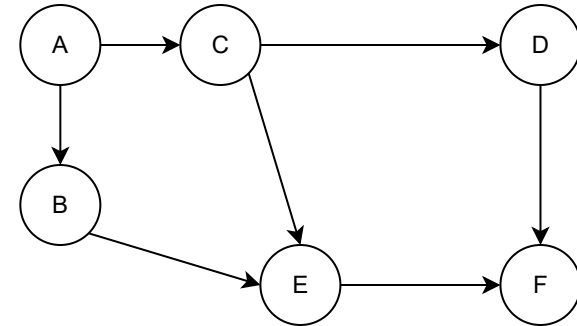
少しずつ問題を小さくしていく (DECREASE AND CONQUER)

(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



トポロジカルソート (TOPOLOGICAL SORT)

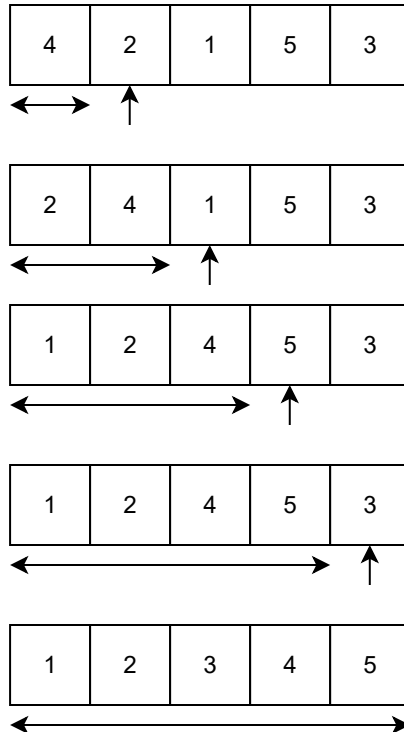


残り4つの整数から1つ取り、ソート済み配列にマージ。
残り3つの整数から1つ取り、ソート済み配列にマージ。
残り2つの整数から1つ取り、ソート済み配列にマージ。
残り1つの整数から1つ取り、ソート済み配列にマージ。

少しずつ問題を小さくしていく (DECREASE AND CONQUER)

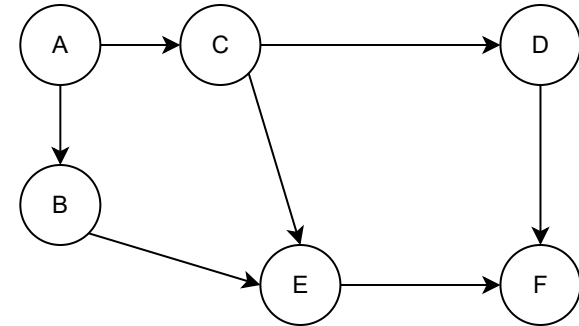
(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



残り4つの整数から1つ取り、ソート済み配列にマージ。
残り3つの整数から1つ取り、ソート済み配列にマージ。
残り2つの整数から1つ取り、ソート済み配列にマージ。
残り1つの整数から1つ取り、ソート済み配列にマージ。

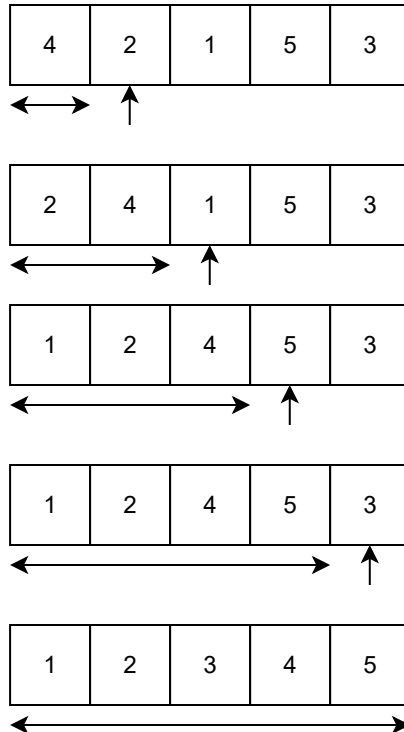
トポロジカルソート (TOPOLOGICAL SORT)



少しずつ問題を小さくしていく (DECREASE AND CONQUER)

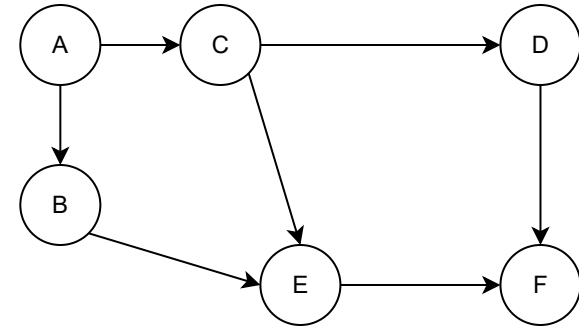
(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



残り4つの整数から1つ取り、ソート済み配列にマージ。
残り3つの整数から1つ取り、ソート済み配列にマージ。
残り2つの整数から1つ取り、ソート済み配列にマージ。
残り1つの整数から1つ取り、ソート済み配列にマージ。

トポロジカルソート (TOPOLOGICAL SORT)

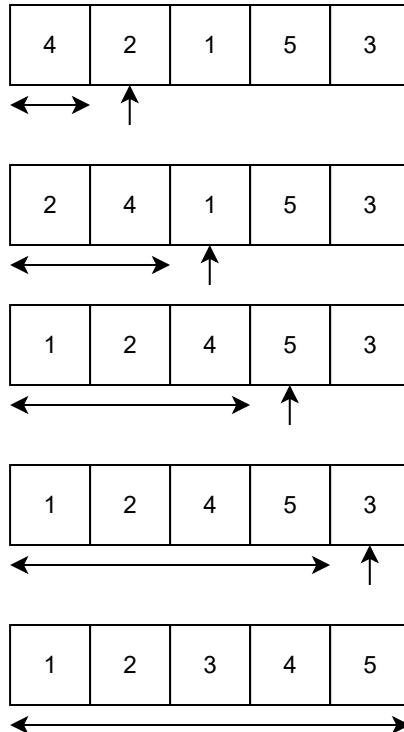


残り6つから実行可能なものを1つ取り、出る枝を削除。

少しずつ問題を小さくしていく (DECREASE AND CONQUER)

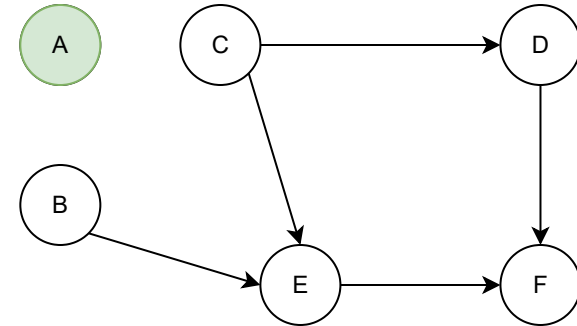
(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



残り4つの整数から1つ取り、ソート済み配列にマージ。
残り3つの整数から1つ取り、ソート済み配列にマージ。
残り2つの整数から1つ取り、ソート済み配列にマージ。
残り1つの整数から1つ取り、ソート済み配列にマージ。

トポロジカルソート (TOPOLOGICAL SORT)

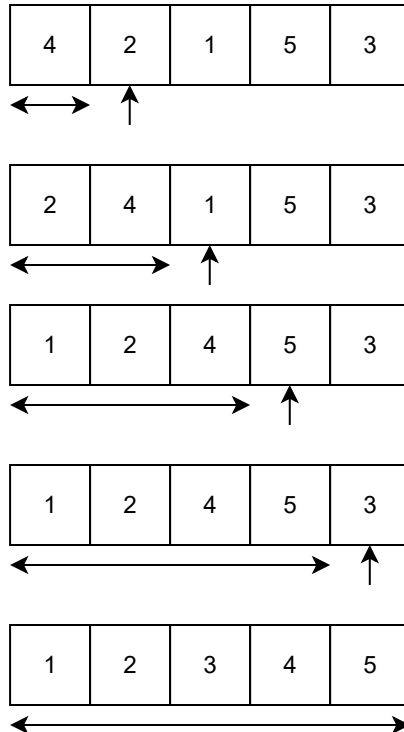


残り6つから実行可能なものを1つ取り、出る枝を削除。
残り5つから実行可能なものを1つ取り、出る枝を削除。

少しずつ問題を小さくしていく (DECREASE AND CONQUER)

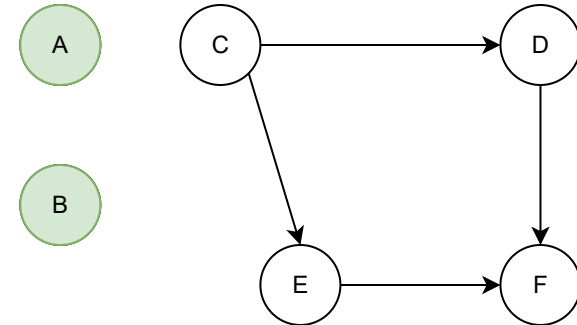
(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



残り4つの整数から1つ取り、ソート済み配列にマージ。
残り3つの整数から1つ取り、ソート済み配列にマージ。
残り2つの整数から1つ取り、ソート済み配列にマージ。
残り1つの整数から1つ取り、ソート済み配列にマージ。

トポロジカルソート (TOPOLOGICAL SORT)

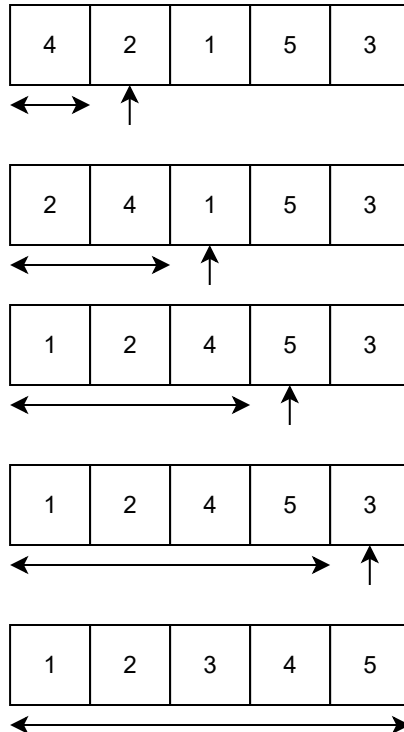


残り6つから実行可能なものを1つ取り、出る枝を削除。
残り5つから実行可能なものを1つ取り、出る枝を削除。
残り4つから実行可能なものを1つ取り、出る枝を削除。

少しずつ問題を小さくしていく (DECREASE AND CONQUER)

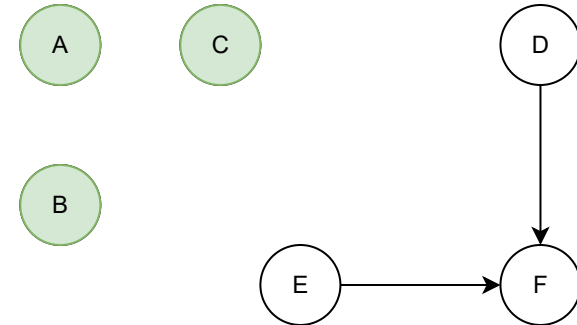
(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



残り4つの整数から1つ取り、ソート済み配列にマージ。
残り3つの整数から1つ取り、ソート済み配列にマージ。
残り2つの整数から1つ取り、ソート済み配列にマージ。
残り1つの整数から1つ取り、ソート済み配列にマージ。

トポロジカルソート (TOPOLOGICAL SORT)

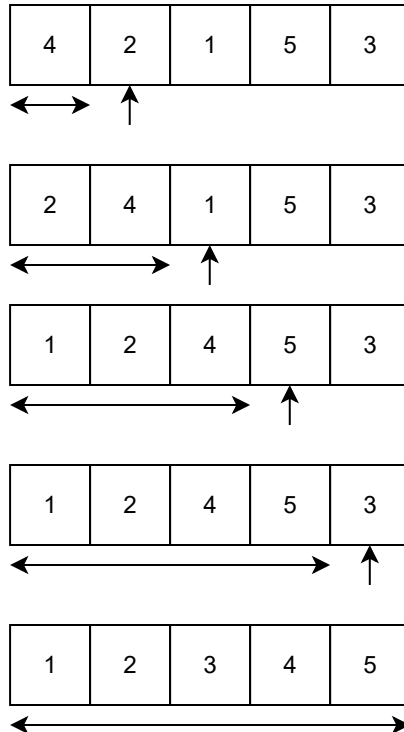


残り6つから実行可能なものを1つ取り、出る枝を削除。
残り5つから実行可能なものを1つ取り、出る枝を削除。
残り4つから実行可能なものを1つ取り、出る枝を削除。
残り3つから実行可能なものを1つ取り、出る枝を削除。

少しずつ問題を小さくしていく (DECREASE AND CONQUER)

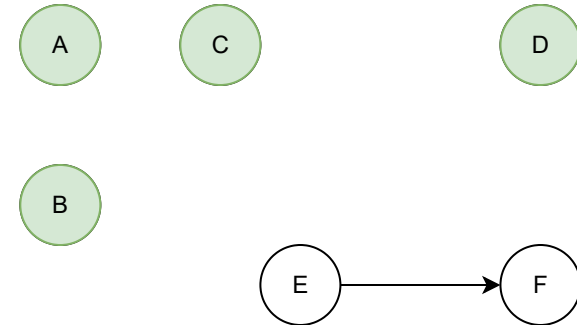
(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



残り4つの整数から1つ取り、ソート済み配列にマージ。
残り3つの整数から1つ取り、ソート済み配列にマージ。
残り2つの整数から1つ取り、ソート済み配列にマージ。
残り1つの整数から1つ取り、ソート済み配列にマージ。

トポロジカルソート (TOPOLOGICAL SORT)

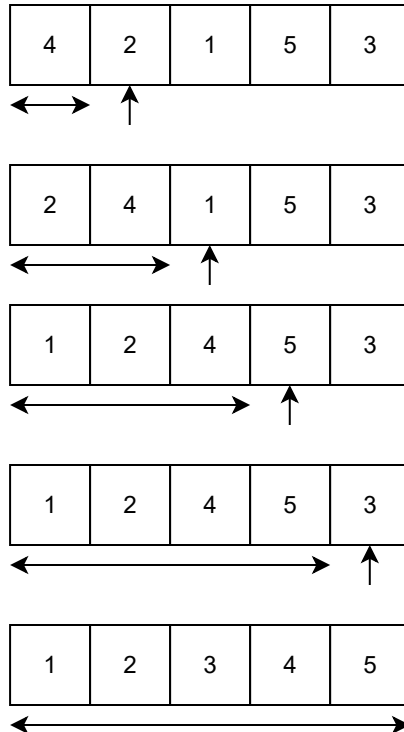


残り6つから実行可能なものを1つ取り、出る枝を削除。
残り5つから実行可能なものを1つ取り、出る枝を削除。
残り4つから実行可能なものを1つ取り、出る枝を削除。
残り3つから実行可能なものを1つ取り、出る枝を削除。
残り2つから実行可能なものを1つ取り、出る枝を削除。

少しずつ問題を小さくしていく (DECREASE AND CONQUER)

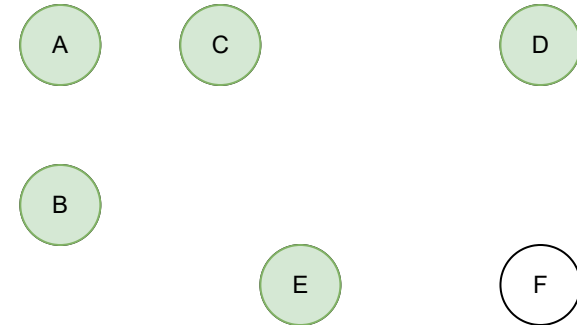
(決まった数だけ小さくしていく)

挿入ソート (INSERTION SORT)



残り4つの整数から1つ取り、ソート済み配列にマージ。
残り3つの整数から1つ取り、ソート済み配列にマージ。
残り2つの整数から1つ取り、ソート済み配列にマージ。
残り1つの整数から1つ取り、ソート済み配列にマージ。

トポロジカルソート (TOPOLOGICAL SORT)



残り6つから実行可能なものを1つ取り、出る枝を削除。
残り5つから実行可能なものを1つ取り、出る枝を削除。
残り4つから実行可能なものを1つ取り、出る枝を削除。
残り3つから実行可能なものを1つ取り、出る枝を削除。
残り2つから実行可能なものを1つ取り、出る枝を削除。
残り1つから実行可能なものを1つ取り、出る枝を削除。

少しずつ問題を小さくしていく (DECREASE AND CONQUER)

(決まった割合だけ小さくしていく)

二分探索 (BINARY SEARCH)

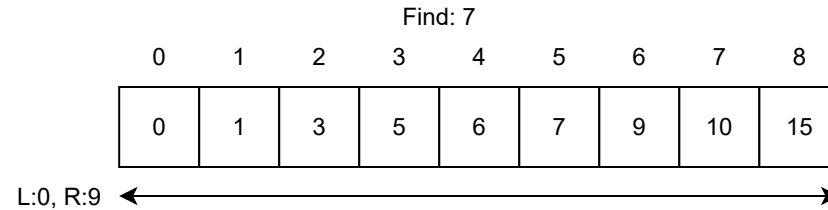
Find: 7

0	1	2	3	4	5	6	7	8
0	1	3	5	6	7	9	10	15

少しずつ問題を小さくしていく (DECREASE AND CONQUER)

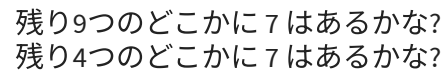
(決まった割合だけ小さくしていく)

二分探索 (BINARY SEARCH)

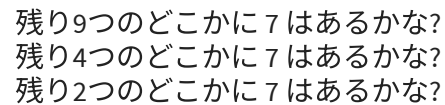


残り9つのどこかに7はあるかな?

二分探索 (BINARY SEARCH)



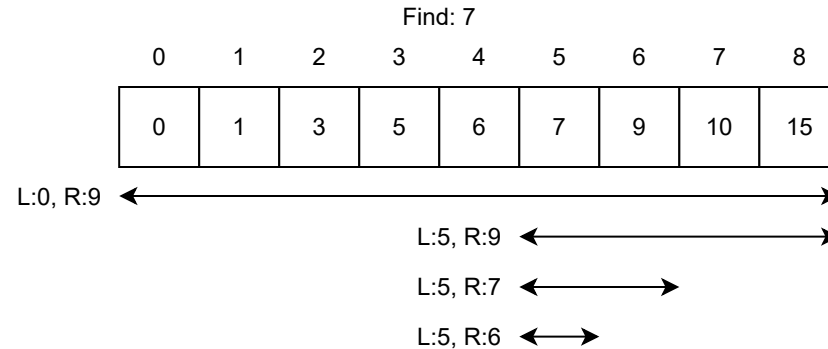
二分探索 (BINARY SEARCH)



少しずつ問題を小さくしていく (DECREASE AND CONQUER)

(決まった割合だけ小さくしていく)

二分探索 (BINARY SEARCH)



残り9つのどこかに7はあるかな?

残り4つのどこかに7はあるかな?

残り2つのどこかに7はあるかな?

残り1つのどこかに7はあるかな?

少しずつ問題を小さくしていく (DECREASE AND CONQUER)

(決まった数じゃないけど小さくしていく)

最大公約数 (Greatest Common Divisor) を求める ユークリッドの互除法。

$$\gcd(x, y) = \begin{cases} y & (x \text{ が } y \text{ で割り切れる時}) \\ \gcd(y, x \% y) & (\text{それ以外の時}) \end{cases}$$

$$\begin{aligned} \gcd(305, 110) &= \gcd(110, 85) \\ &= \gcd(85, 25) \\ &= \gcd(25, 10) \\ &= \gcd(10, 5) \\ &= 5 \end{aligned}$$

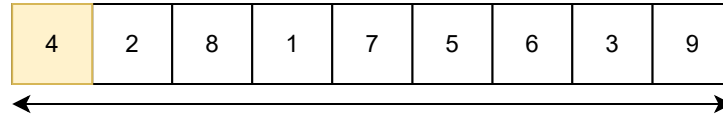
複数の部分問題に分割 (DIVIDE AND CONQUER)

クイックソート (QUICK SORT)

4	2	8	1	7	5	6	3	9
---	---	---	---	---	---	---	---	---

複数の部分問題に分割 (DIVIDE AND CONQUER)

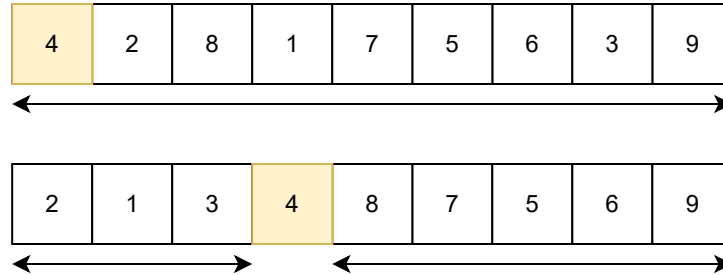
クイックソート (QUICK SORT)



1つ数字を選んで(4)それより小さい、大きいで分ける。

複数の部分問題に分割 (DIVIDE AND CONQUER)

クイックソート (QUICK SORT)

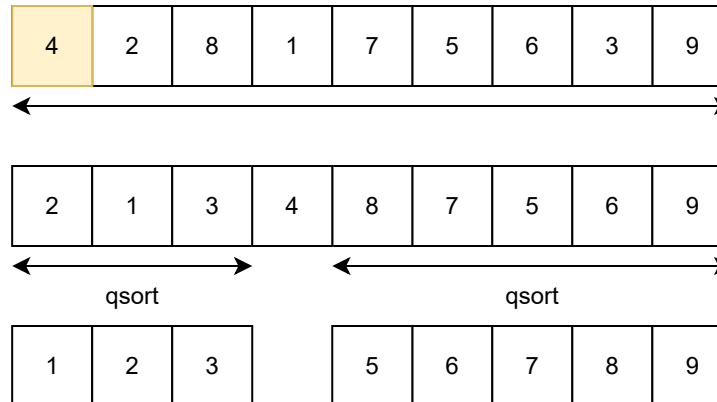


1つ数字を選んで(4)それより小さい、大きいで分ける。
4以上、4以下の組が出来た。

Diagram illustrating the partitioning step of Quicksort. The top array is [4, 2, 8, 1, 7, 5, 6, 3, 9]. The pivot is 4. The bottom array shows the result after partitioning: [2, 1, 3, 4, 8, 7, 5, 6, 9]. The pivot 4 is now in its sorted position. The subarrays [2, 1, 3] and [8, 7, 5, 6, 9] are marked for recursive sorting with 'qsort' labels.

10.3

クイックソート (QUICK SORT)



1つ数字を選んで(4)それより小さい、大きいで分ける。

4以上、4以下の組が出来た。

後は各々 quick sort しておいてください。

完成

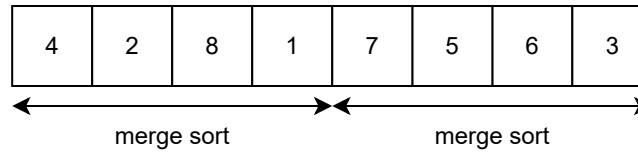
複数の部分問題に分割 (DIVIDE AND CONQUER)

マージソート (MERGE SORT)

4	2	8	1	7	5	6	3
---	---	---	---	---	---	---	---

複数の部分問題に分割 (DIVIDE AND CONQUER)

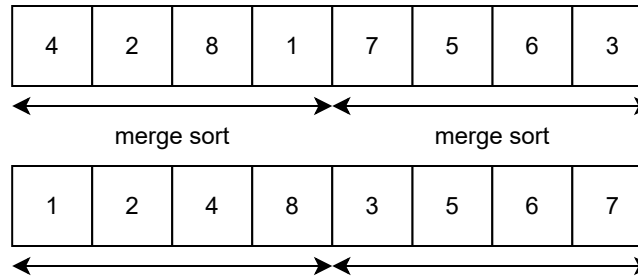
マージソート (MERGE SORT)



まずは半分ずつマージソート

複数の部分問題に分割 (DIVIDE AND CONQUER)

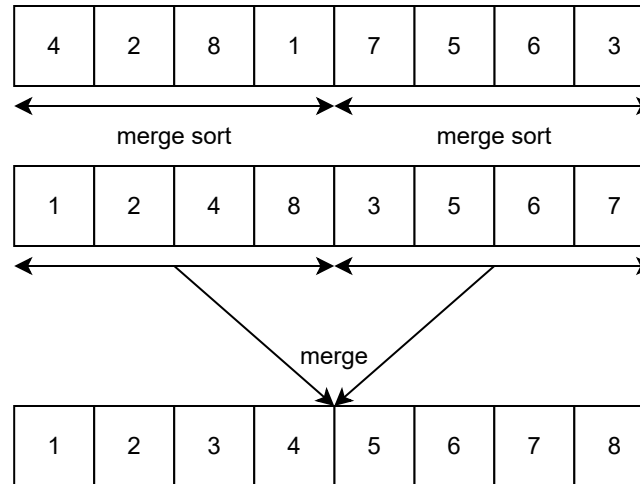
マージソート (MERGE SORT)



まずは半分ずつマージソート
2つのソート済み配列が出来た。

複数の部分問題に分割 (DIVIDE AND CONQUER)

マージソート (MERGE SORT)



まずは半分ずつマージソート
2つのソート済み配列が出来た。
マージして完成。

複数の部分問題に分割 (DIVIDE AND CONQUER)

$n \times n$ 行列の掛け算 $C = AB$ を計算したい。

$$A = \begin{pmatrix} a_{00} & a_{01} & \cdots & a_{0n} \\ a_{10} & a_{11} & \cdots & a_{1n} \\ & & \vdots & \\ a_{n0} & a_{n1} & \cdots & a_{nn} \end{pmatrix}$$

のように表現すると、

$C = AB$ を計算するには、

$$c_{xy} = a_{x0}b_{0y} + a_{x1}b_{1y} + \cdots + a_{xn}b_{ny}$$

の計算を $0 \leq x, y \leq n$ に対して計算するので、
計算量は $O(n^3)$

複数の部分問題に分割 (DIVIDE AND CONQUER)

シュトラッセンのアルゴリズム (STRASSEN'S ALGORITHM)

行列の掛け算を $O(n^3) = O(n^{(\log_2 8)})$ ではなく、 $O(n^{2.807}) = O(n^{(\log_2 7)})$ で計算する。

$$C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

と分割して、

$$P_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$C_{11} = P_1 + P_4 - P_5 + P_7$$

$$P_2 = (A_{21} + A_{22})B_{11}$$

$$C_{12} = P_3 + P_5$$

$$P_3 = A_{11}(B_{12} - B_{22})$$

$$C_{21} = P_2 + P_4$$

$$P_4 = A_{22}(B_{21} - B_{11})$$

$$C_{22} = P_1 + P_3 - P_2 + P_6$$

$$P_5 = (A_{11} + A_{12})B_{22}$$

$$P_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$P_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

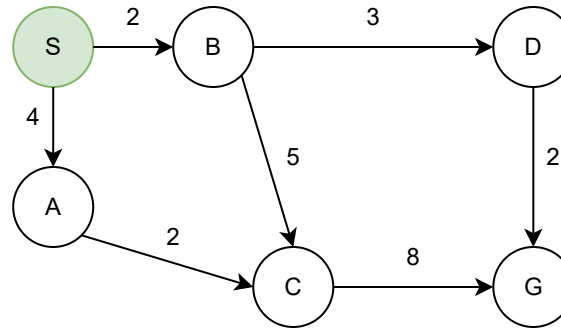
とすると、

半分サイズの行列の掛け算 7 回で行列を計算できる。

貪欲法

目の前の良さそうな選択肢を採用する

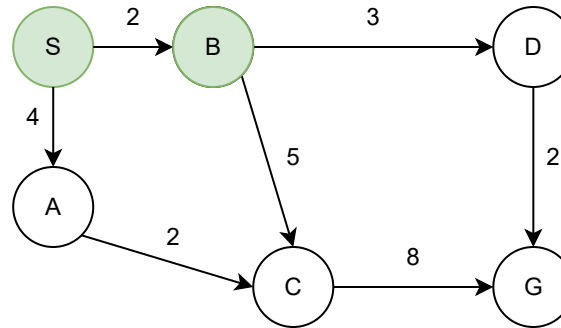
最短距離 - ダイクストラ法(DIJKSTRA'S)



到達可能な点と距離は B:2, A:4。

目の前の良さそうな選択肢を採用する

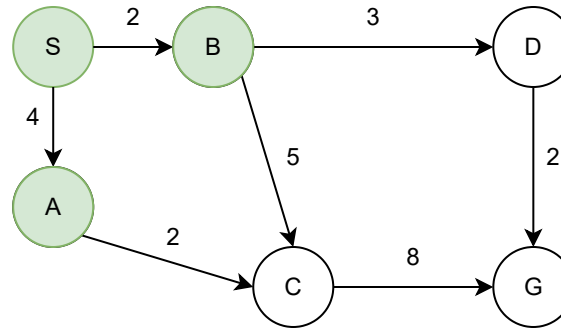
最短距離 - ダイクストラ法(DIJKSTRA'S)



到達可能な点と距離は B:2, A:4。
一番近い B を採用。到達可能な点と距離は A:4, D:5, C: 7。

目の前の良さそうな選択肢を採用する

最短距離 - ダイクストラ法(DIJKSTRA'S)



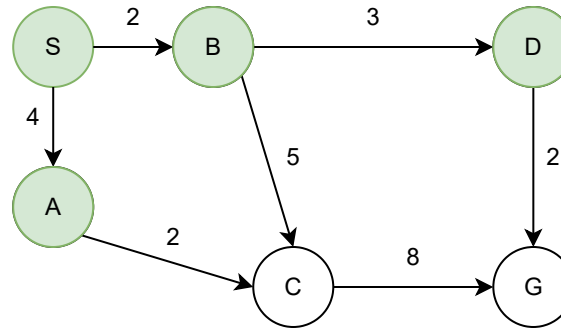
到達可能な点と距離は B:2, A:4。

一番近い B を採用。到達可能な点と距離は A:4, D:5, C: 7。

一番近い A を採用。到達可能な点と距離は D:5, C:6。

目の前の良さそうな選択肢を採用する

最短距離 - ダイクストラ法(DIJKSTRA'S)



到達可能な点と距離は B:2, A:4。

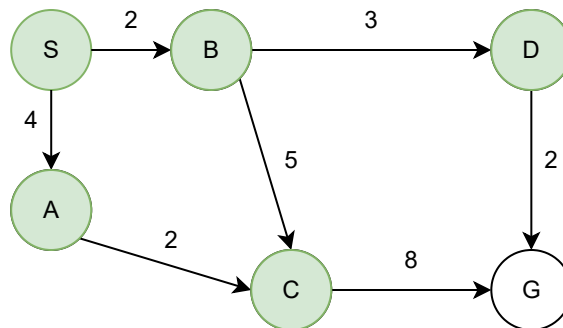
一番近い B を採用。到達可能な点と距離は A:4, D:5, C: 7。

一番近い A を採用。到達可能な点と距離は D:5, C:6。

一番近い D を採用。到達可能な点と距離は C:6, G:7。

目の前の良さそうな選択肢を採用する

最短距離 - ダイクストラ法(DIJKSTRA'S)



到達可能な点と距離は B:2, A:4。

一番近い B を採用。到達可能な点と距離は A:4, D:5, C: 7。

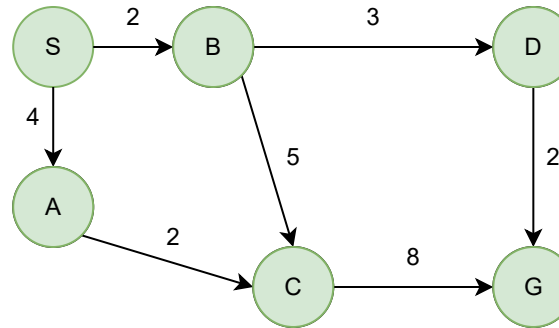
一番近い A を採用。到達可能な点と距離は D:5, C:6。

一番近い D を採用。到達可能な点と距離は C:6, G:7。

一番近い C を採用。到達可能な点と距離は G:7。

目の前の良さそうな選択肢を採用する

最短距離 - ダイクストラ法(DIJKSTRA'S)



到達可能な点と距離は B:2, A:4。

一番近い B を採用。到達可能な点と距離は A:4, D:5, C:7。

一番近い A を採用。到達可能な点と距離は D:5, C:6。

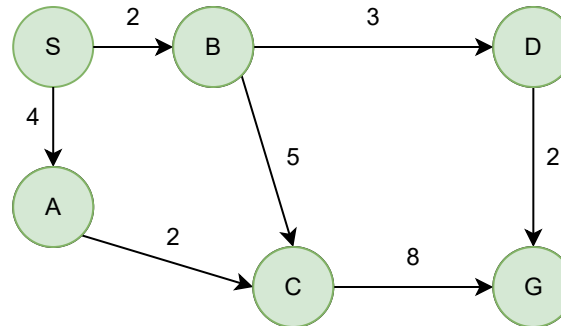
一番近い D を採用。到達可能な点と距離は C:6, G:7。

一番近い C を採用。到達可能な点と距離は G:7。

一番近い G を採用。G までの距離は 7。

目の前の良さそうな選択肢を採用する

最短距離 - ダイクストラ法(DIJKSTRA'S)



到達可能な点と距離は B:2, A:4。

一番近い B を採用。到達可能な点と距離は A:4, D:5, C: 7。

一番近い A を採用。到達可能な点と距離は D:5, C:6。

一番近い D を採用。到達可能な点と距離は C:6, G:7。

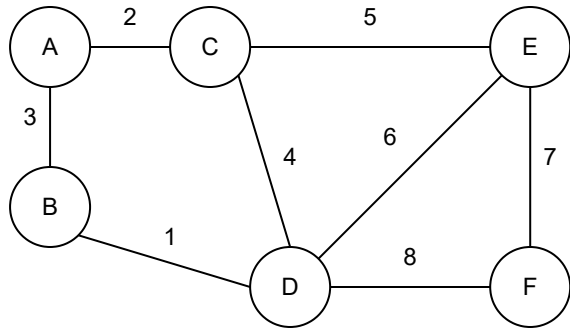
一番近い C を採用。到達可能な点と距離は G:7。

一番近い G を採用。G までの距離は 7。

負の辺があると、貪欲法(ダイクストラ)で最短となるとは限らない。

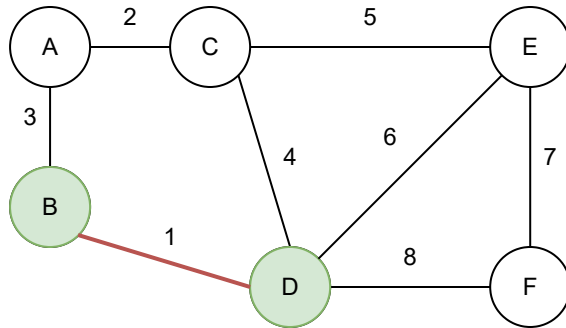
目の前の良さそうな選択肢を採用する

最小全域木 - クラスカル法 (KRUSKAL'S)



目の前の良さそうな選択肢を採用する

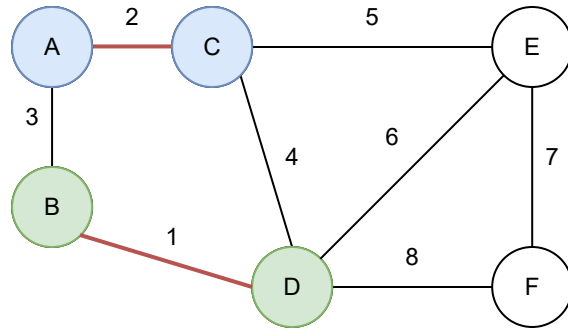
最小全域木 - クラスカル法 (KRUSKAL'S)



一番安くて(コスト1) 木を大きくする B-D 採用

目の前の良さそうな選択肢を採用する

最小全域木 - クラスカル法 (KRUSKAL'S)

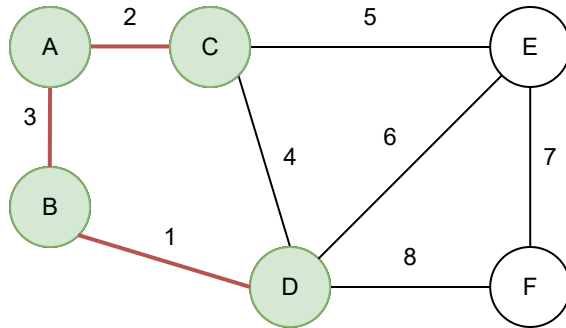


一番安くて(コスト1) 木を大きくする B-D 採用

次に安くて(コスト2) 木を大きくする A-C 採用

目の前の良さそうな選択肢を採用する

最小全域木 - クラスカル法 (KRUSKAL'S)



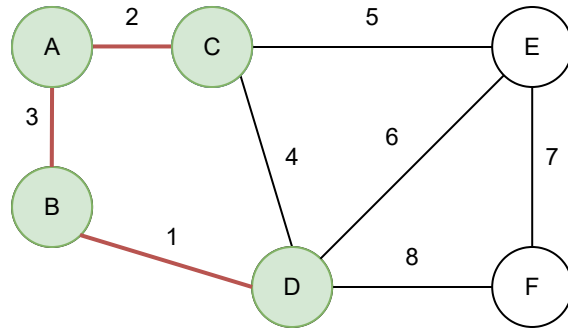
一番安くて(コスト1) 木を大きくする B-D 採用

次に安くて(コスト2) 木を大きくする A-C 採用

次に安くて(コスト3) 木を大きくする A-B 採用

目の前の良さそうな選択肢を採用する

最小全域木 - クラスカル法 (KRUSKAL'S)



一番安くて(コスト1) 木を大きくする B-D 採用

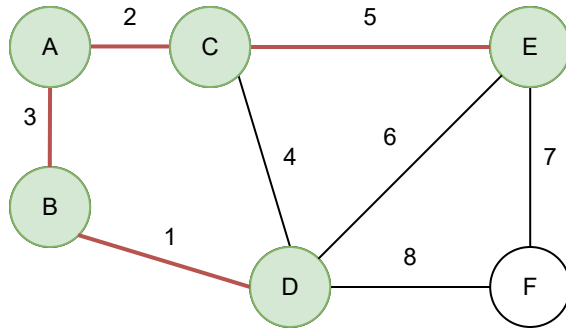
次に安くて(コスト2) 木を大きくする A-C 採用

次に安くて(コスト3) 木を大きくする A-B 採用

次に安い(コスト4) C-D は木を大きくしない

目の前の良さそうな選択肢を採用する

最小全域木 - クラスカル法 (KRUSKAL'S)



一番安くて(コスト1) 木を大きくする B-D 採用

次に安くて(コスト2) 木を大きくする A-C 採用

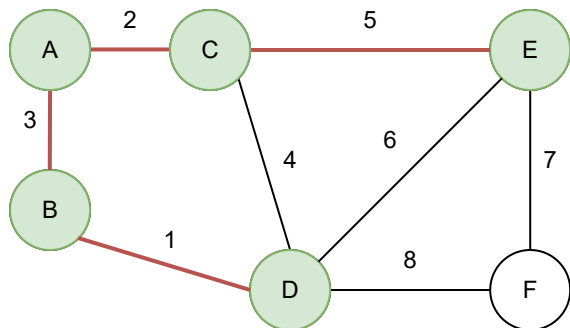
次に安くて(コスト3) 木を大きくする A-B 採用

次に安い(コスト4) C-D は木を大きくしない

次に安くて(コスト5) 木を大きくする C-E 採用

目の前の良さそうな選択肢を採用する

最小全域木 - クラスカル法 (KRUSKAL'S)



一番安くて(コスト1) 木を大きくする B-D 採用

次に安くて(コスト2) 木を大きくする A-C 採用

次に安くて(コスト3) 木を大きくする A-B 採用

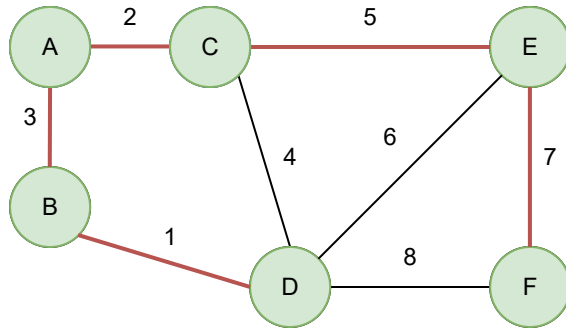
次に安い(コスト4) C-D は木を大きくしない

次に安くて(コスト5) 木を大きくする C-E 採用

次に安い(コスト6) D-E は木を大きくしない

目の前の良さそうな選択肢を採用する

最小全域木 - クラスカル法 (KRUSKAL'S)



一番安くて(コスト1) 木を大きくする B-D 採用

次に安くて(コスト2) 木を大きくする A-C 採用

次に安くて(コスト3) 木を大きくする A-B 採用

次に安い(コスト4) C-D は木を大きくしない

次に安くて(コスト5) 木を大きくする C-E 採用

次に安い(コスト6) D-E は木を大きくしない

次に安くて(コスト7) 木を大きくする E-F 採用

目の前の良さそうな選択肢を採用する

ナップサック問題

$w=10$ にできるだけ価値を詰め込みたい。

	w	v	v/w
銀	3	9	3.33
鉄	8	20	2.50
銅	9	22	2.44

比重 (v/w) の大きいものから詰め込んでいく。

目の前の良さそうな選択肢を採用する

ナップサック問題

$w=10$ にできるだけ価値を詰め込みたい。

	w	v	v/w
銀	3	9	3.33
鉄	8	20	2.50
銅	9	22	2.44

比重 (v/w) の大きいものから詰め込んでいく。

早く計算はできるが、最適な解にはならない

問題の変換

問題の変換 (TRANSFORM AND CONQUER)

ソート済み配列から重複を発見したい
→ 隣接した各二要素が同じかチェック

優先度付きキュー (heap) にデータを全部入れて全部(小さい方から)取り出す
→ ソートできた(heap sort)

AとBの最小公倍数 (least common multiple) $LCM(A, B)$ を求めたい
→ $LCM(A, B) = A \times B / GCD(A, B)$

線形計画法 (LINEAR PROGRAMMING)

複数の線形制約式が与えられた時に、線形目的関数を最大化する問題。

一次式・線形式

$a_1x_1 + a_2x_2 + \cdots + a_nx_n \leq b$ みたいな形の式。 ($\leq$ は $=$ でも構わない)

$a_1, a_2, \dots, a_n, b$ が定数。 $x_1, x_2, \dots, x_n$ が変数。

左辺が、変数が高々1つ掛けられた項の和になっている。

様々な問題が線形計画法に変更出来て便利。

(というか、様々な問題が線形計画法になってしまうので研究されてきた。)

後述する simplex 法などを用いて解くことができる。

問題の変換 (TRANSFORM AND CONQUER)

線形計画法 (LINEAR PROGRAMMING)

この町のお店には、1個10円のチョコが8個。1個5円のバナナが10個売っています。
貴方はチョコ1つとバナナ1つでチョコバナナを作ることが出来ますが、疲れるので3つまでしか作れません。
隣町のお店では、チョコを1個15円。バナナを1個3円。チョコバナナを1個30円で買ってくれます。
仕入れに100円まで使えるとき、隣町に出かける前に何をどれだけ買えば利益が最大化できますか？

目的関数	$15x_{\text{チョコ}} + 3x_{\text{バナナ}} + 30x_{\text{チョコバナナ}} + x_{\text{おつり}}$
------	---

制約式(仕入れ)	$10(x_{\text{チョコ}} + x_{\text{チョコバナナ}})$ $+ 5(x_{\text{バナナ}} + x_{\text{チョコバナナ}}) + x_{\text{おつり}} = 100$
----------	--

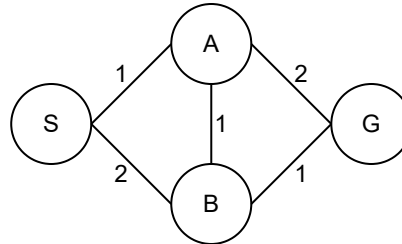
制約式(売切)	$x_{\text{チョコ}} + x_{\text{チョコバナナ}} \leq 8$ $x_{\text{バナナ}} + x_{\text{チョコバナナ}} \leq 10$
---------	---

制約式(疲れ)	$x_{\text{チョコバナナ}} \leq 3$
---------	----------------------------

非負制約	$x_{\text{チョコ}}, x_{\text{バナナ}}, x_{\text{チョコバナナ}}, x_{\text{おつり}} \geq 0$
------	--

線形計画法 (LINEAR PROGRAMMING)

SからGまで出来るだけ多くの水を流したいです。数字は、辺に流せる水の最大容量です。



目的関数

$$x_{SA} + x_{SB}$$

制約式(容量)

$$x_{SA} \leq 1, x_{SB} \leq 2$$

$$x_{AB} \leq 1, x_{BA} \leq 1$$

$$x_{AG} \leq 2, x_{BG} \leq 1$$

制約式(中間点)

$$x_{SA} + x_{BA} - x_{AG} - x_{AB} = 0$$

$$x_{SB} + x_{AB} - x_{BG} - x_{BA} = 0$$

非負制約

$$x_{SA}, x_{SB}, x_{AB}, x_{BA}, x_{AG}, x_{BG} \geq 0$$

動的計画法

ある計算が、自分自身の部分問題の計算に依存し、
部分問題の計算を再利用して効率的に求める手法

ある計算が、自分自身の部分問題の計算に依存し、
部分問題の計算を再利用して効率的に求める手法

最長共通部分文字列(LCS: LONGEST COMMON SUBSEQUENCE)

二つの文字列(A, B)の共通の部分の最長の長さを求めたい。
A: 「あんぱん」とB: 「あぱまん」と、「あぱん」で3文字。

	あ	ん	ぱ	ん
あ	1	1	1	1
ぱ	1	1	2	2
ま	1	1	2	2
ん	1	2	2	3

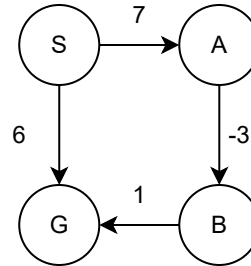
LCS(a,b) を文字列 Aの先頭a文字と、Bの先頭b文字の LCS とすると、

$$LCS(a, b) = \begin{cases} 0 & (a = 0 \text{ or } b = 0) \\ LCS(a - 1, b - 1) + 1 & \text{Aのa文字目} = \text{Bのb文字目} \\ \text{MAX}(LCS(a - 1, b), \\ \quad LCS(a, b - 1)) & (otherwise) \end{cases}$$

ワーシャルフロイド法(FLOYD-WARSHALL ALGORITHM)

全点对の最短距離を求めるアルゴリズム。

```
for (int k = 0; k < n; k++)  
  for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
      path[i][j] = min(path[i][j], path[i][k] + path[k][j]);
```



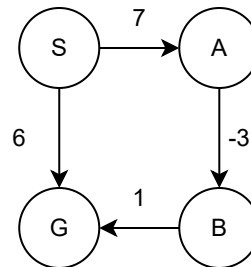
寄道を許さない全点間の最短距離テーブル(つまり直線距離)から、
点 0 への寄道を許した全点間の最短距離テーブルを $k=0$ で作り、
上記テーブルから、点 0, 1 への寄道を許した全点間の最短距離テーブルを $k=1$ で作り、
上記テーブルから、点 0, 1, 2 への寄道を許した全点間の最短距離テーブルを $k=2$ で作り、
最終的に点 0, 1, ..., n への寄道が許される最短距離テーブルを構築している。

ベルマンフォード法(BELLMAN-FORD ALGORITHM)

ある点から各点への最短距離を求めるアルゴリズム。

```
int dist[nV]; // nV は点の数
for (int i = 0; i < nV; i++)
    dist[i] = (i == S ? 0 : INF); // S はスタート点

for (int i = 0; i < nV-1; i++)
    for (int j = 0; j < nE; j++) // nE は辺の数
        dist[E[j].to] = min(dist[E[j].to],
                             dist[E[j].from] + E[j].weight);
```



Sから枝を0本以下使っての辿り着ける範囲の最短距離テーブルから、
Sから枝を1本以下使っての辿り着ける範囲の最短距離テーブルを作り、
上記から、Sから枝を2本以下使っての辿り着ける範囲の最短距離テーブルを作り、
上記から、Sから枝を3本以下使っての辿り着ける範囲の最短距離テーブルを作り、
最終的に、Sから枝をV-1本以下使って辿り着ける
(つまり好きな点に寄道可能な)最短距離テーブルを計算している。

ベルマンフォード法(BELLMAN-FORD ALGORITHM)

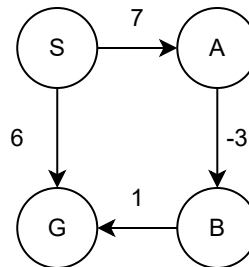
ある点から各点への最短距離を求めるアルゴリズム。

```
int dist[nV]; // nV は点の数
for (int i = 0; i < nV; i++)
    dist[i] = (i == S ? 0 : INF); // S はスタート点

for (int i = 0; i < nV-1; i++)
    for (int j = 0; j < nE; j++) // nE は辺の数
        dist[E[j].to] = min(dist[E[j].to],
                             dist[E[j].from] + E[j].weight);
```

```
//dist[x][y] は、 x個までの枝を使う Sからyまでの最短距離
int dist[nV][nV];
for (int j = 0; j < nV; j++)
    for (int i = 0; i < nV; i++)
        dist[j][i] = ((j == 0 && i == S) ? 0 : INF);

for (int i = 1; i < nV; i++)
    for (int j = 0; j < nE; j++) // nE は辺の数
        dist[i][E[j].to] = min(min(dist[i-1][E[j].to],
                                     dist[i][E[j].to]),
                                dist[i-1][E[j].from] + E[j].weight);
```



Sから枝を0本以下使っての辿り着ける範囲の最短距離テーブルから、
Sから枝を1本以下使っての辿り着ける範囲の最短距離テーブルを作り、
上記から、Sから枝を2本以下使っての辿り着ける範囲の最短距離テーブルを作り、
上記から、Sから枝を3本以下使っての辿り着ける範囲の最短距離テーブルを作り、
最終的に、Sから枝をV-1本以下使って辿り着ける
(つまり好きな点に寄道可能な)最短距離テーブルを計算している。

時間と空間のトレードオフ

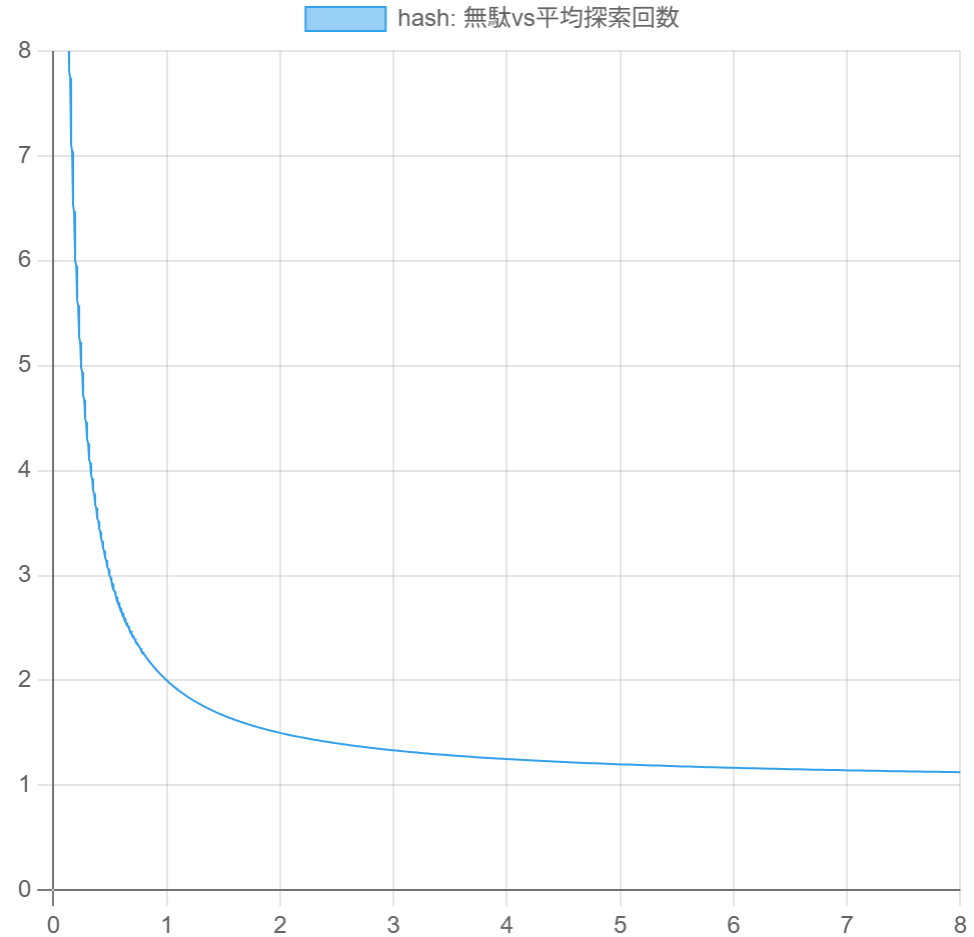
ハッシュテーブル

0	1	2	3	4	5	6	7
	17						

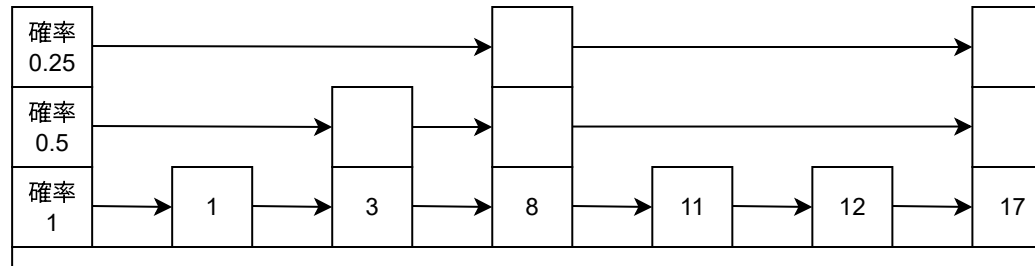
Hash(17) = 17 % 8 = 1

Hash(A) % Capacity (Capacity: 容量) の場所にデータを
保管

p = Size / Capacity が、
50% (無駄: $(1-0.5) / 0.5 = 100\%$)だと、
平均探索回数 2回
80% (無駄: $(1-0.8) / 0.8 = 25\%$)だと、
平均探索回数 5回
99.9% (無駄: $1-0.999 / 0.999 = 0.1\%$)だと、
平均探索回数 1,000回



スキップリスト



探索は、最上段リストで範囲を絞り込み、
下のリストで同じことを繰り返す。

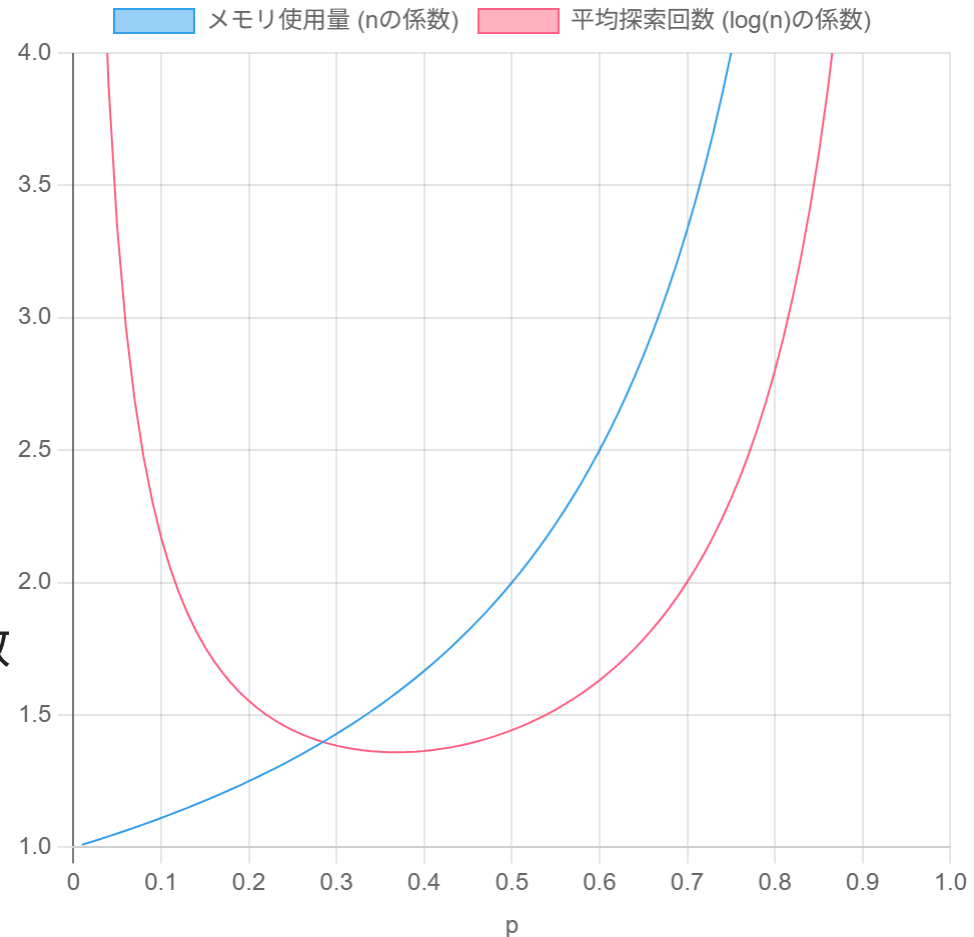
挿入は、探索後に最下段に挿入、
確率 p で下から2段目に連結。
更に確率 p で...と繰り返し上段に挿入。

必要なメモリは、最下段+2段目+...

$$= n + np + np^2 + \dots = \frac{1}{1-p} \times n$$

平均探索回数は、各段の平均探索回数×段数

$$= \frac{1}{2p} \times \log_{\frac{1}{p}} n = \frac{1}{2p \log \frac{1}{p}} \log n$$



近似アルゴリズム

近似アルゴリズム

巡回セールスマン問題、ナップサック問題の最適解を計算するのは難しい。
(NP完全問題 = 多分 $O(2^n)$)

厳密解を諦めて、そこまで悪くないアルゴリズムが欲しい。

ナップサック問題

巡回セールスマン問題

近似アルゴリズム

巡回セールスマン問題、ナップサック問題の最適解を計算するのは難しい。
(NP完全問題 = 多分 $O(2^n)$)

厳密解を諦めて、そこまで悪くないアルゴリズムが欲しい。

ナップサック問題

近似値2 (厳密解より2倍以上悪くならない) の貪欲法+が知られている。
近似解: 貪欲法で解いた答えと、一番価値のある品物、の大きい方

巡回セールスマン問題

近似アルゴリズム

巡回セールスマン問題、ナップサック問題の最適解を計算するのは難しい。
(NP完全問題 = 多分 $O(2^n)$)

厳密解を諦めて、そこまで悪くないアルゴリズムが欲しい。

ナップサック問題

近似値2 (厳密解より2倍以上悪くならない) の貪欲法+が知られている。
近似解: 貪欲法で解いた答えと、一番価値のある品物、の大きい方

巡回セールスマン問題

クリストフィードのアルゴリズム

近似度1.5。距離が三角不等式を満たしている (つまり、 $A \rightarrow C \rightarrow B$ が $A \rightarrow B$ より近くなるCは無い)時のみ利用可能。

逐次改善法

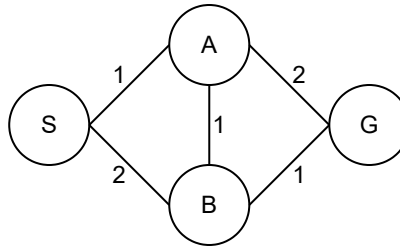
逐次改善法 (ITERATIVE IMPROVEMENT)

一つ答えを見つけ、それより良い答えに改善を繰り返す手法。

最適解が見つかることが保証されている方法も、そうでない方法 (近似解手法) もある。

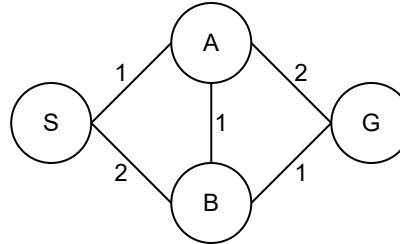
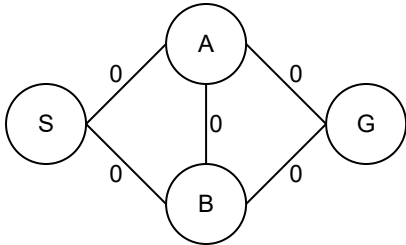
逐次改善法 (ITERATIVE IMPROVEMENT)

フォードファルカーソンのアルゴリズム (FORD-FULKERSON)



逐次改善法 (ITERATIVE IMPROVEMENT)

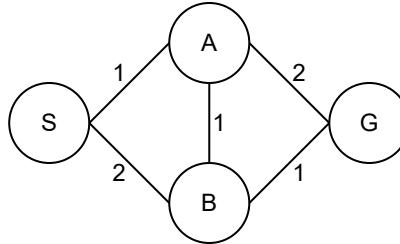
フォードファルカーソンのアルゴリズム (FORD-FULKERSON)



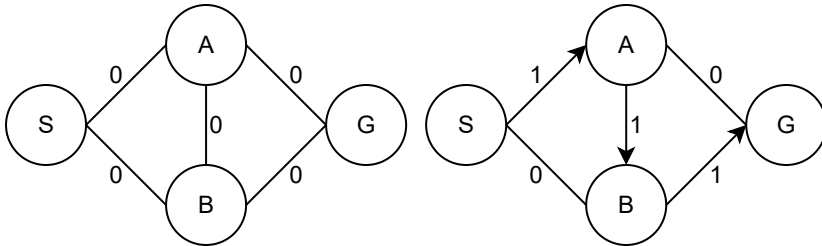
現在分かっている範囲の最適解として流れ0を用意。
DFSなどで、一つ水を流す方法を見つける。

逐次改善法 (ITERATIVE IMPROVEMENT)

フォードファルカーソンのアルゴリズム (FORD-FULKERSON)

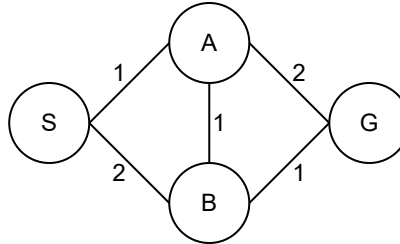


現在分かっている範囲の最適解として流れ0を用意。
DFSなどで、一つ水を流す方法を見つける。
1の流れが見つかった。

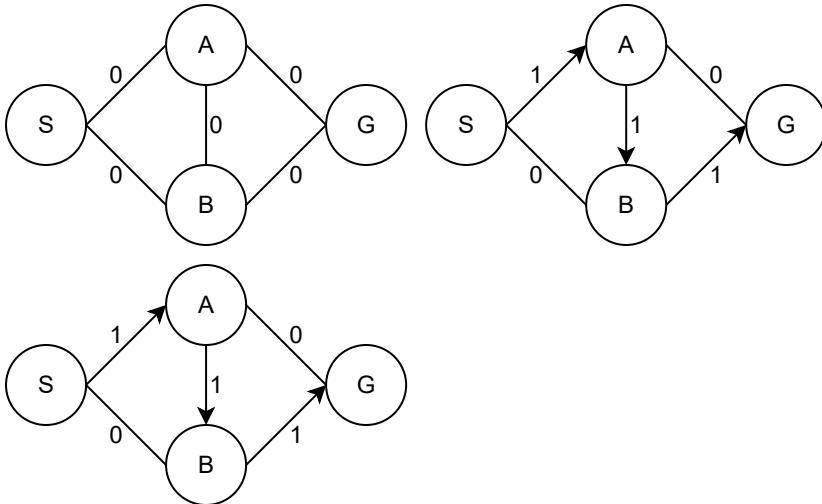


逐次改善法 (ITERATIVE IMPROVEMENT)

フォードファルカーソンのアルゴリズム (FORD-FULKERSON)

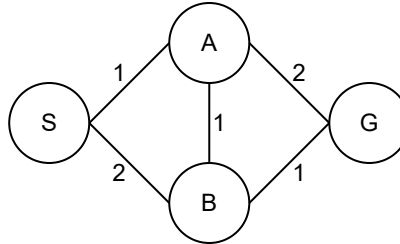


現在分かっている範囲の最適解として流れ0を用意。
 DFSなどで、一つ水を流す方法を見つける。
 1の流れが見つかった。
 これを採用し、現在の最適解にマージ。

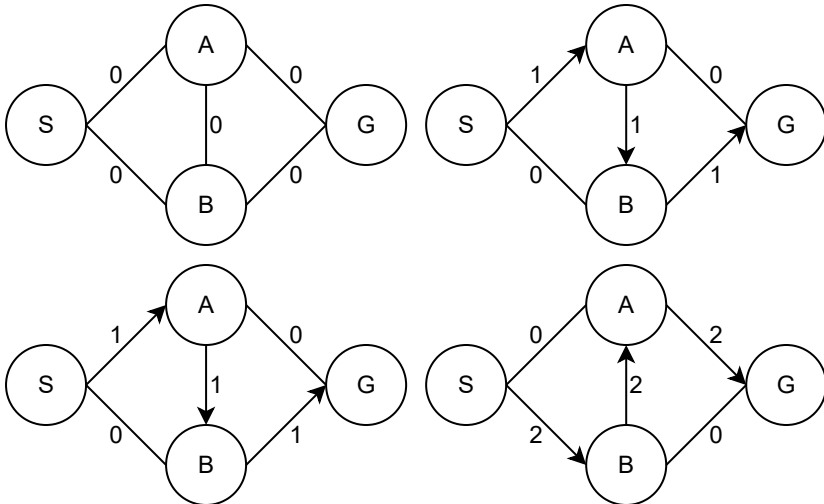


逐次改善法 (ITERATIVE IMPROVEMENT)

フォードファルカーソンのアルゴリズム (FORD-FULKERSON)

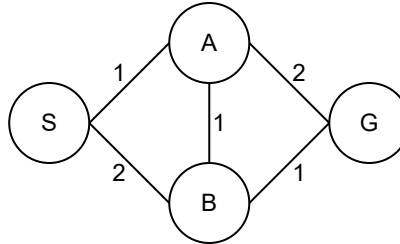


現在分かっている範囲の最適解として流れ0を用意。
 DFSなどで、一つ水を流す方法を見つける。
 1の流れが見つかった。
 これを採用し、現在の最適解にマージ。
 次は2の流れが見つかった。

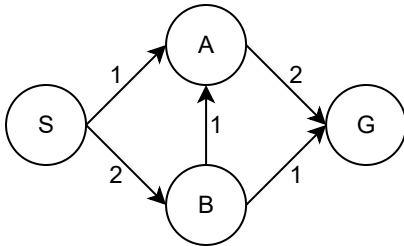
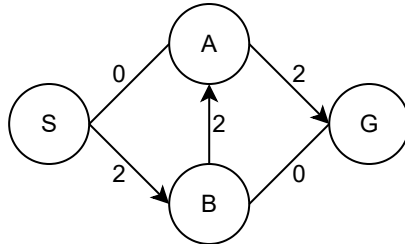
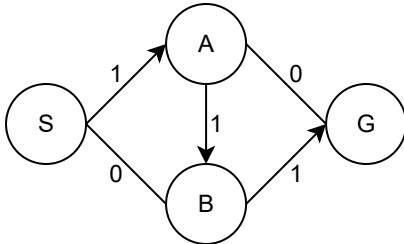
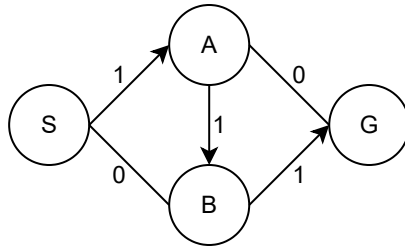
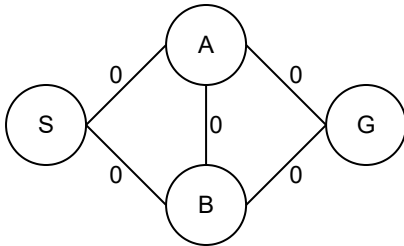


逐次改善法 (ITERATIVE IMPROVEMENT)

フォードファルカーソンのアルゴリズム (FORD-FULKERSON)

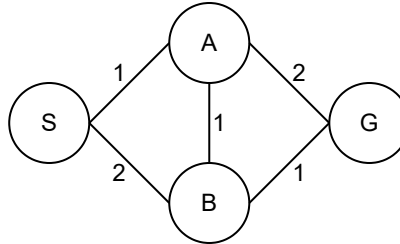


現在分かっている範囲の最適解として流れ0を用意。
DFSなどで、一つ水を流す方法を見つける。
1の流れが見つかった。
これを採用し、現在の最適解にマージ。
次は2の流れが見つかった。
これを最適解にマージ。



逐次改善法 (ITERATIVE IMPROVEMENT)

フォードファルカーソンのアルゴリズム (FORD-FULKERSON)



現在分かっている範囲の最適解として流れ0を用意。

DFSなどで、一つ水を流す方法を見つける。

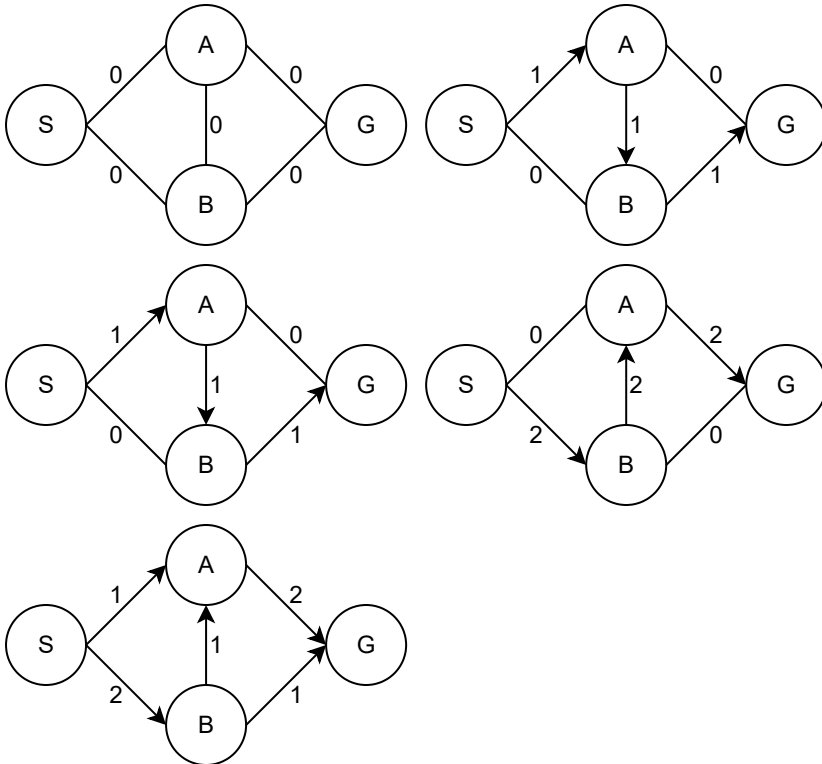
1の流れが見つかった。

これを採用し、現在の最適解にマージ。

次は2の流れが見つかった。

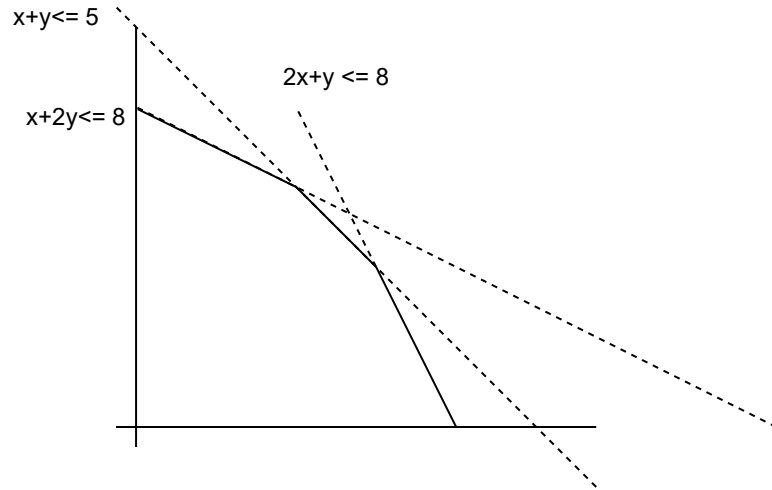
これを最適解にマージ。

これ以上見つからないので答えは3



逐次改善法 (ITERATIVE IMPROVEMENT)

シンプレックス法 (SIMPLEX)

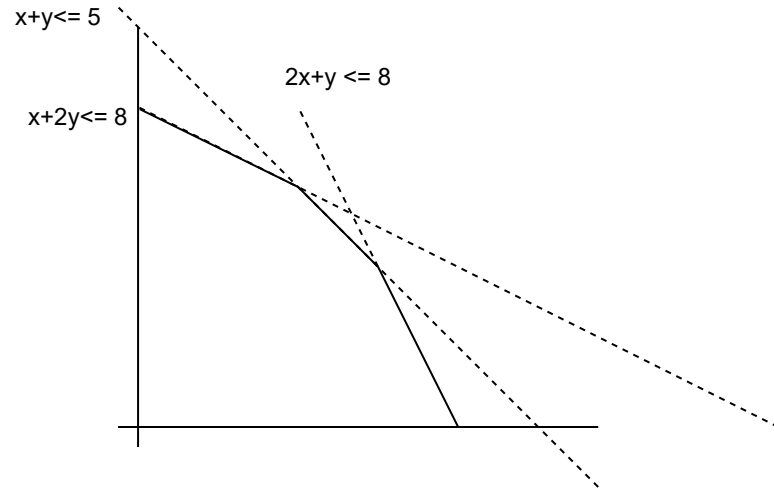


目的関数: $f(x, y) = 3x + 2y$

制約: $x + 2y \leq 8, x + y \leq 5, 2x + y \leq 8$

逐次改善法 (ITERATIVE IMPROVEMENT)

シンプレックス法 (SIMPLEX)



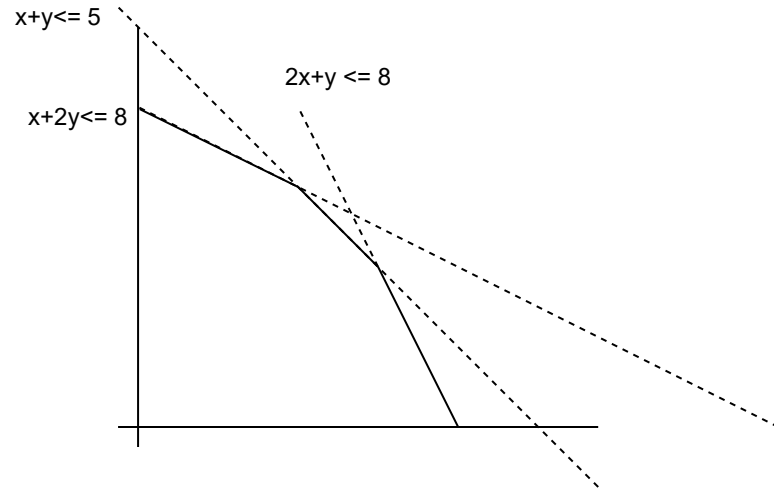
まずは実行可能な点 $(0,0)$ からスタート。

目的関数: $f(x, y) = 3x + 2y$

制約: $x + 2y \leq 8, x + y \leq 5, 2x + y \leq 8$

逐次改善法 (ITERATIVE IMPROVEMENT)

シンプレックス法 (SIMPLEX)



まずは実行可能な点 $(0,0)$ からスタート。

$(4,0)$ と $(0,4)$ が隣接している。

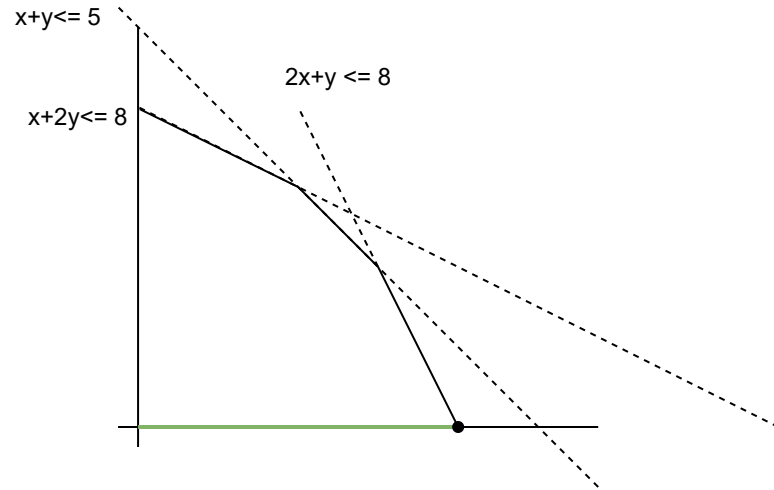
$f(0,0) = 0, f(4,0) = 12, f(0,4) = 8$
 なので、 $(4,0)$ に移動。

目的関数: $f(x, y) = 3x + 2y$

制約: $x + 2y \leq 8, x + y \leq 5, 2x + y \leq 8$

逐次改善法 (ITERATIVE IMPROVEMENT)

シンプレックス法 (SIMPLEX)



まずは実行可能な点 $(0,0)$ からスタート。

$(4,0)$ と $(0,4)$ が隣接している。

$f(0,0) = 0, f(4,0) = 12, f(0,4) = 8$
 なので、 $(4,0)$ に移動。

次は $(3,2)$ に隣接している。

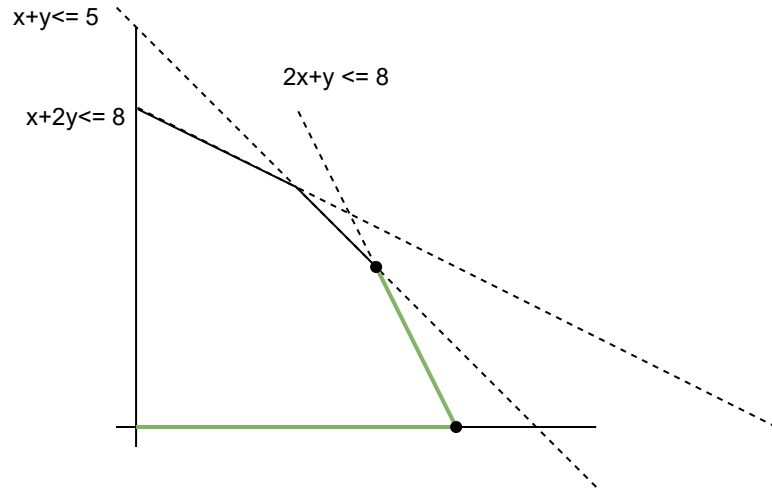
$f(3,2) = 13 > f(4,0) = 12$
 なので、 $(3,2)$ に移動。

目的関数: $f(x, y) = 3x + 2y$

制約: $x + 2y \leq 8, x + y \leq 5, 2x + y \leq 8$

逐次改善法 (ITERATIVE IMPROVEMENT)

シンプレックス法 (SIMPLEX)



まずは実行可能な点 $(0,0)$ からスタート。

$(4,0)$ と $(0,4)$ が隣接している。

$f(0,0) = 0, f(4,0) = 12, f(0,4) = 8$
 なので、 $(4,0)$ に移動。

次は $(3,2)$ に隣接している。

$f(3,2) = 13 > f(4,0) = 12$
 なので、 $(3,2)$ に移動。

次は $(2,3)$ に隣接している。

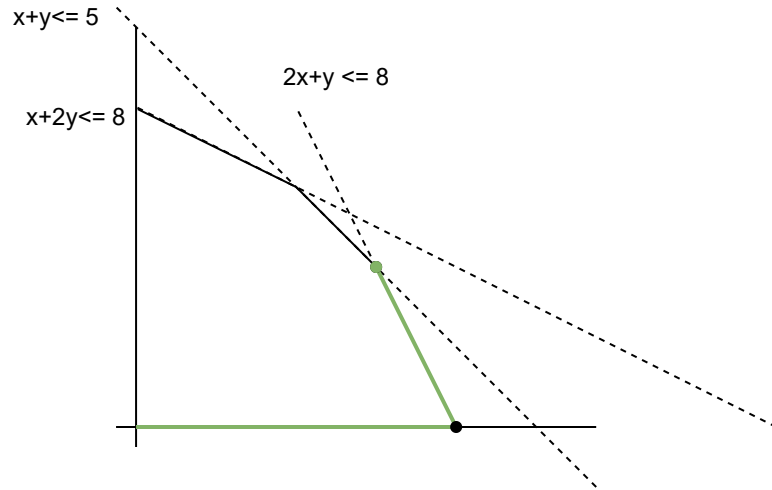
しかし、今いる $f(3,2)$ の方が $f(2,3)$ より大きい。

目的関数: $f(x, y) = 3x + 2y$

制約: $x + 2y \leq 8, x + y \leq 5, 2x + y \leq 8$

逐次改善法 (ITERATIVE IMPROVEMENT)

シンプレックス法 (SIMPLEX)



まずは実行可能な点 $(0,0)$ からスタート。

$(4,0)$ と $(0,4)$ が隣接している。

$f(0,0) = 0, f(4,0) = 12, f(0,4) = 8$
 なので、 $(4,0)$ に移動。

次は $(3,2)$ に隣接している。

$f(3,2) = 13 > f(4,0) = 12$
 なので、 $(3,2)$ に移動。

次は $(2,3)$ に隣接している。

しかし、今いる $f(3,2)$ の方が $f(2,3)$ より大きい。

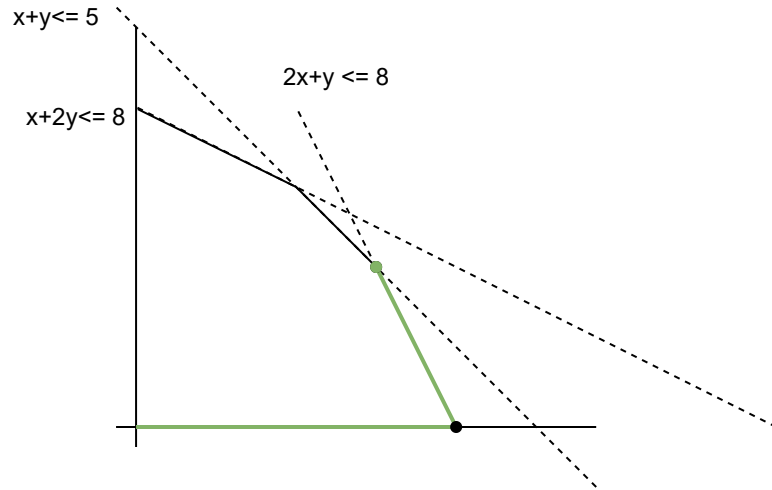
最適解は $(3,2)$

目的関数: $f(x, y) = 3x + 2y$

制約: $x + 2y \leq 8, x + y \leq 5, 2x + y \leq 8$

逐次改善法 (ITERATIVE IMPROVEMENT)

シンプレックス法 (SIMPLEX)



目的関数: $f(x, y) = 3x + 2y$

制約: $x + 2y \leq 8, x + y \leq 5, 2x + y \leq 8$

まずは実行可能な点 $(0,0)$ からスタート。

$(4,0)$ と $(0,4)$ が隣接している。
 $f(0,0) = 0, f(4,0) = 12, f(0,4) = 8$
 なので、 $(4,0)$ に移動。

次は $(3,2)$ に隣接している。
 $f(3,2) = 13 > f(4,0) = 12$
 なので、 $(3,2)$ に移動。

次は $(2,3)$ に隣接している。
 しかし、今いる $f(3,2)$ の方が $f(2,3)$ より大きい。

最適解は $(3,2)$

悪質な例だと指数時間かかるが、
 実用上問題無く高速。

確率的アルゴリズム

確率的アルゴリズム

最大カット問題

グラフの点を2グループに分け、グループ間を跨る枝の数を最大化する問題 (NP困難)
各点がどちらのグループかをランダムに割り当てると、期待値が全枝の半分となる。

BALLS AND BINS

n 個の箱に n 個のボールをランダムに入れる。(1個ずつ独立にランダムに選ぶ)。
この時、一番多くボールが入っている箱のボールの数は、

$1 - \frac{1}{n^{c-2+o(1)}}$ の確率で (つまり $c > 2$ で n が十分大きいとほぼ1で)、 $c \frac{\log n}{\log \log n}$ 以下になる。

ラビンミラー素数判定法

ある奇数 n が素数が調べたい。 (3以上の素数は全て奇数)

$n - 1 = 2^s d$ (d は奇数)として、 $1 < a < n$ となる a をランダムに選ぶ。

$a^d \not\equiv 1 \pmod{n}$ かつ、 $0 \leq x < s$ の全 x で、 $a^{2^x d} \not\equiv -1 \pmod{n}$ なら、合成数。

そうでないなら、素数の可能性有り。

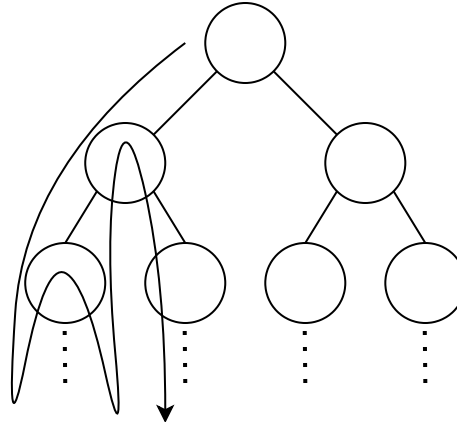
n が合成数なら、一つの a でテストすると $3/4$ 以上の確率で合成数と判定できるので、
十分な数の a をランダムにテストし、合成数判定とならなければ、素数と見做す。

(確実なAKS素数判定より早いので良く使われる。($O(\log(n)^{7.5})$ vs $O(k \log(n)^3)$))

指数的爆発対策

指数的爆発対策

バックトラック / DFS



途中の深さでの計算結果を覚えておき、
左の子供の計算が終わった後に、右の子供の計算の為に再利用する。

しかし深さが $2, 3, 4, \dots$ と増えると、探索数が $4, 8, 16, \dots 2^n$ と爆発的に増える。

何とかしたい。

指数的爆発対策

A*探索アルゴリズム

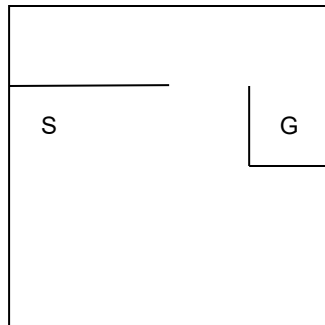
効率的に最短距離を求めるアルゴリズム。

$f(n) = g(n) + h(n)$ として、 $f(n)$ が小さいノードから順番に探索。

$g(n)$ はスタートから n への最短距離。 $h(n)$ は n からゴールへの距離の予測。

$h^*(n)$ が n からゴールの真の距離として、

常に $0 \leq h(n) \leq h^*(n)$ なら (控えめに見積もっておけば)、A*は最適解を返す。



$h(n)$ として、Gとのx,y軸の差(マンハッタン距離)を使う。

指数的爆発対策

A*探索アルゴリズム

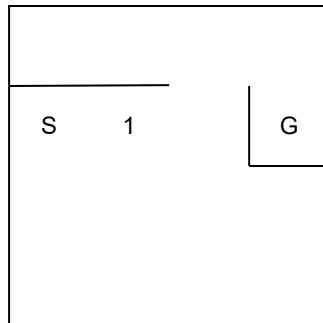
効率的に最短距離を求めるアルゴリズム。

$f(n) = g(n) + h(n)$ として、 $f(n)$ が小さいノードから順番に探索。

$g(n)$ はスタートから n への最短距離。 $h(n)$ は n からゴールへの距離の予測。

$h^*(n)$ が n からゴールの真の距離として、

常に $0 \leq h(n) \leq h^*(n)$ なら (控えめに見積もっておけば)、A*は最適解を返す。



$h(n)$ として、Gとのx,y軸の差(マンハッタン距離)を使う。

1の場所は、 $f(n) = 1 + 2 = 3$

指数的爆発対策

A*探索アルゴリズム

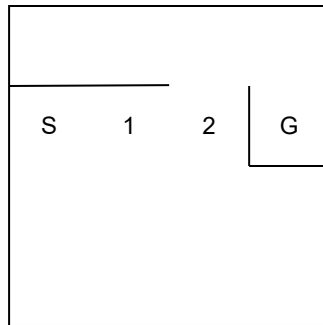
効率的に最短距離を求めるアルゴリズム。

$f(n) = g(n) + h(n)$ として、 $f(n)$ が小さいノードから順番に探索。

$g(n)$ はスタートから n への最短距離。 $h(n)$ は n からゴールへの距離の予測。

$h^*(n)$ が n からゴールの真の距離として、

常に $0 \leq h(n) \leq h^*(n)$ なら (控えめに見積もっておけば)、A*は最適解を返す。



$h(n)$ として、Gとのx,y軸の差(マンハッタン距離)を使う。

1の場所は、 $f(n) = 1 + 2 = 3$

2の場所は、 $f(n) = 2 + 1 = 3$

コスト3でいける場所が無くなった。

指数的爆発対策

A*探索アルゴリズム

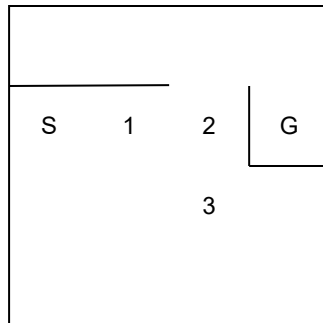
効率的に最短距離を求めるアルゴリズム。

$f(n) = g(n) + h(n)$ として、 $f(n)$ が小さいノードから順番に探索。

$g(n)$ はスタートから n への最短距離。 $h(n)$ は n からゴールへの距離の予測。

$h^*(n)$ が n からゴールの真の距離として、

常に $0 \leq h(n) \leq h^*(n)$ なら (控えめに見積もっておけば)、A*は最適解を返す。



$h(n)$ として、Gとのx,y軸の差(マンハッタン距離)を使う。

1の場所は、 $f(n) = 1 + 2 = 3$

2の場所は、 $f(n) = 2 + 1 = 3$

コスト3でいける場所が無くなった。

3の場所は、 $f(n) = 3 + 2 = 5$

指数的爆発対策

A*探索アルゴリズム

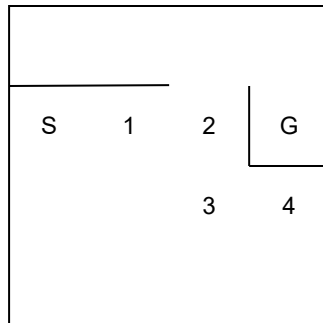
効率的に最短距離を求めるアルゴリズム。

$f(n) = g(n) + h(n)$ として、 $f(n)$ が小さいノードから順番に探索。

$g(n)$ はスタートから n への最短距離。 $h(n)$ は n からゴールへの距離の予測。

$h^*(n)$ が n からゴールの真の距離として、

常に $0 \leq h(n) \leq h^*(n)$ なら (控えめに見積もっておけば)、A*は最適解を返す。



$h(n)$ として、Gとのx,y軸の差(マンハッタン距離)を使う。

1の場所は、 $f(n) = 1 + 2 = 3$

2の場所は、 $f(n) = 2 + 1 = 3$

コスト3でいける場所が無くなった。

3の場所は、 $f(n) = 3 + 2 = 5$

4の場所は、 $f(n) = 4 + 1 = 5$

この周辺にコスト5はもうない。

指数的爆発対策

A*探索アルゴリズム

効率的に最短距離を求めるアルゴリズム。

$f(n) = g(n) + h(n)$ として、 $f(n)$ が小さいノードから順番に探索。

$g(n)$ はスタートから n への最短距離。 $h(n)$ は n からゴールへの距離の予測。

$h^*(n)$ が n からゴールの真の距離として、

常に $0 \leq h(n) \leq h^*(n)$ なら (控えめに見積もっておけば)、 A^* は最適解を返す。

				5
S	1	2	G	
		3		4

$h(n)$ として、Gとのx,y軸の差(マンハッタン距離)を使う。

1の場所は、 $f(n) = 1 + 2 = 3$

2の場所は、 $f(n) = 2 + 1 = 3$

コスト3でいける場所が無くなった。

3の場所は、 $f(n) = 3 + 2 = 5$

4の場所は、 $f(n) = 4 + 1 = 5$

この周辺にコスト5はもうない。

5の場所は、 $f(n) = 3 + 2 = 5$

指数的爆発対策

A*探索アルゴリズム

効率的に最短距離を求めるアルゴリズム。

$f(n) = g(n) + h(n)$ として、 $f(n)$ が小さいノードから順番に探索。

$g(n)$ はスタートから n への最短距離。 $h(n)$ は n からゴールへの距離の予測。

$h^*(n)$ が n からゴールの真の距離として、

常に $0 \leq h(n) \leq h^*(n)$ なら (控えめに見積もっておけば)、 A^* は最適解を返す。

		5	6
S	1	2	G
		3	4

$h(n)$ として、G との x, y 軸の差 (マンハッタン距離) を使う。

1の場所は、 $f(n) = 1 + 2 = 3$

2の場所は、 $f(n) = 2 + 1 = 3$

コスト3でいける場所が無くなった。

3の場所は、 $f(n) = 3 + 2 = 5$

4の場所は、 $f(n) = 4 + 1 = 5$

この周辺にコスト5はもうない。

5の場所は、 $f(n) = 3 + 2 = 5$

6の場所は、 $f(n) = 4 + 1 = 5$

指数的爆発対策

A*探索アルゴリズム

効率的に最短距離を求めるアルゴリズム。

$f(n) = g(n) + h(n)$ として、 $f(n)$ が小さいノードから順番に探索。

$g(n)$ はスタートから n への最短距離。 $h(n)$ は n からゴールへの距離の予測。

$h^*(n)$ が n からゴールの真の距離として、

常に $0 \leq h(n) \leq h^*(n)$ なら (控えめに見積もっておけば)、A*は最適解を返す。

		5	6
S	1	2	7
		3	4
			G

$h(n)$ として、Gとのx,y軸の差(マンハッタン距離)を使う。

1の場所は、 $f(n) = 1 + 2 = 3$

2の場所は、 $f(n) = 2 + 1 = 3$

コスト3でいける場所が無くなった。

3の場所は、 $f(n) = 3 + 2 = 5$

4の場所は、 $f(n) = 4 + 1 = 5$

この周辺にコスト5はもうない。

5の場所は、 $f(n) = 3 + 2 = 5$

6の場所は、 $f(n) = 4 + 1 = 5$

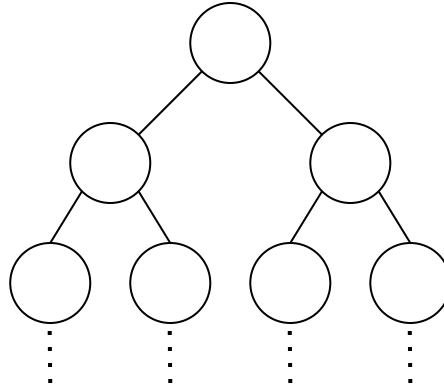
7の場所は、 $f(n) = 5 + 0 = 5$

答えが見つかった。

指数的爆発対策

分枝限定法

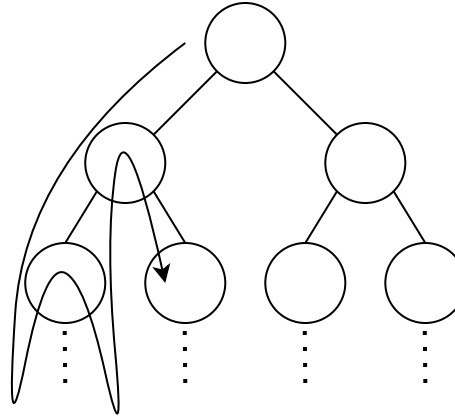
探索の途中で、正解の下限に達する見込の無い枝を捨てていく。



指数的爆発対策

分枝限定法

探索の途中で、正解の下限に達する見込みの無い枝を捨てていく。



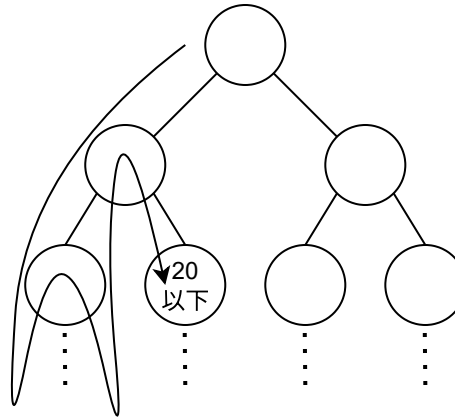
答えは 30以上

探索の途中で、正解が30以上である選択枝を発見。

指数的爆発対策

分枝限定法

探索の途中で、正解の下限に達する見込みの無い枝を捨てていく。



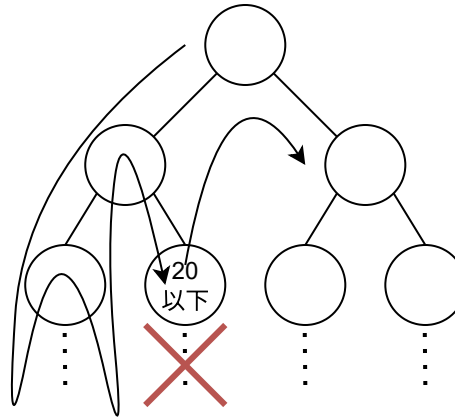
答えは 30以上

探索の途中で、正解が30以上である選択枝を発見。
このノード以下は探索しても20以下のスコアにしかない。

指数的爆発対策

分枝限定法

探索の途中で、正解の下限に達する見込の無い枝を捨てていく。



答えは 30 以上

探索の途中で、正解が30以上である選択枝を発見。
このノード以下は探索しても20以下のスコアにしかない。
探索しても無駄。見ずに次に行く。

ループと再帰

ループで書けるものは再帰でも書ける。逆も然り。

再帰版

```
bool dfs(Node *node, int t) {  
    if (node->left && dfs(node->left, t))  
        return true;  
    if (node->right && dfs(node->right, t))  
        return true;  
    return (node->value == t);  
}
```

ループ版

```
bool dfs(Node *node, int t) {  
    stack<pair<Node *, int>> s;  
    s.push(make_pair(node, LEFT));  
    while (!s.empty()) {  
        pair<Node *, int> p = s.top(); s.pop();  
        if (p.second == LEFT) {  
            s.push(make_pair(node, RIGHT));  
            if (p.first->left)  
                s.push(make_pair(p.first->left, LEFT));  
        } else if (p.second == RIGHT) {  
            s.push(make_pair(node, MIDDLE));  
            if (p.first->right)  
                s.push(make_pair(p.first->right, LEFT));  
        } else {  
            if (p.first->value == t)  
                return true;  
        }  
    }  
    return false;  
}
```